

重庆邮电大学

CHONGQING UNIVERSITY OF POSTS AND TELECOM.

Moss

2 4
2.1 概述 4
2.2 核心设计思想 4
2.3 关键数据结构 5
2.4 进程与线程创建 8
2.5 程序执行 10
2.6 退出与资源回收 11
2.7 进程组与会话 11
2.8 权限与进程控制 12
2.9 本章小结 13
3 15
3.1 概述 15
3.2 任务调度机制 16
3.3 阻塞与唤醒机制 19
3.4 通用阻塞机制的应用 22
3.5 Futex 同步机制 24
3.6 本章小结 26
4 27
4.1 整体架构 27
4.2 地址空间结构 27
4.3 页表与地址翻译 28
4.4 权限管理 29
4.5 地址空间的建立 30
4.6 按需分页与缺页处理 33
4.7 地址空间复制与写时复制 35
4.8 地址空间的解除与释放 36
4.9 本章小结 37
5 38
5.1 整体架构 38
5.2 挂载子系统 38
5.3 VFS 抽象层 39
5.4 文件系统实现 42
5.5 I/O 子系统 45
5.6 文件元数据与权限 48
5.7 文件事件通知 49
5.8 性能对比 51
5.9 本章小结 53
6 54
6.1 StarryOS 的信号机制 54
6.2 进程可暂停状态的引入 55
6.3 进程身份与信号权限校验 56
6.4 父进程对子进程状态的可见性 56
6.5 小结 57
7 58
7.1 StarryOS 的网络栈 58

7.2 地址族支持与兼容性补齐 58
7.3 Buffer 扩展 59
7.4 Loopback 路径优化 60
7.5 性能对比 61
7.6 小结 63
8 QEMU 64
8.1 平台抽象层架构 64
8.2 双架构 QEMU 平台对比 65
8.3 VirtIO 设备驱动 65
8.4 跨平台行为一致性 67

2

进程与线程管理负责组织用户程序的执行主体及其资源，向内连接地址空间、文件描述符和信号等内核对象，向外提供创建、执行、退出、等待和控制接口。

2.1

已经建立了可运行的进程线程框架。每个用户执行流对应一个 Thread，同一进程中的线程共同持有 ProcessData。前者保存线程私有状态，后者保存地址空间、文件资源、进程级信号和父子关系等共享状态。在功能上， 已经提供了 fork、clone、clone3、execve、exit 和 waitpid，能够完成进程创建、程序映像替换、线程退出和僵尸进程回收，进程组、会话和 也已有基本实现。

然而，基线只处理了 创建子进程、运行、退出、父进程回收 这条主路径，接口组合复杂后问题便逐渐暴露： 的标志约束和失败回滚不够严密， 缺少 execveat 及执行权限检查， 无法完整表达停止和继续状态， 部分接口直接返回成功而没有保存身份变化， 和 的观测链路也尚未打通。换句话说，基线能建立进程，却难以把进程的身份、状态和生命周期完全串联起来。

保留了 的主体结构，重点补齐这些接口之间的联系。主要增量如表所示：

内容	基线	的增量
对象与状态	Thread 与 ProcessData 分离	补充凭据、停止 继续状态、子进程时间和元数据
创建与执行	clone3、execve 已实现	补 严 和 失 败 回 滚 ， 增 加 execveat 与执行约束
退出与等待	线程退出、僵尸进程和 waitpid	完善资源清理，增加 waitid、状态事件和等待
管理接口	进程组、会话、基础凭据与	修正全局登记，实现有状态凭据和最小控制链路

表 进程与线程模块的主要增量

2.2

已经搭起了进程线程管理的基本框架， 的工作是在这套框架中进一步理清状态归属和对象关系：哪些数据属于单个线程，哪些资源应由整个进程共享；进程之间如何组织；身份信息怎样参与权限判断；一个任务又该如何从创建走到退出和回收。下面从这几个问题分别说明。

2.2.1

进程和线程并不是两套互不相干的对象。 延续 的做法，将状态分为两层：Thread 表示单个执行流，ProcessData 表示线程共同拥有的进程资源。创建普通线程时复用 ProcessData；创建新进程时再根据 标志复制或共享其中的资源。

这种划分回答了一个最基本的问题：某项状态究竟属于线程，还是属于整个进程。线程名称、线程信号状态和退出字段放在 Thread 中；地址空间、程序路径、文件表作用域、凭据和子进程集合放在 ProcessData 中。只有边界清楚， 、 和 才不会各自维护一份互相冲突的状态。

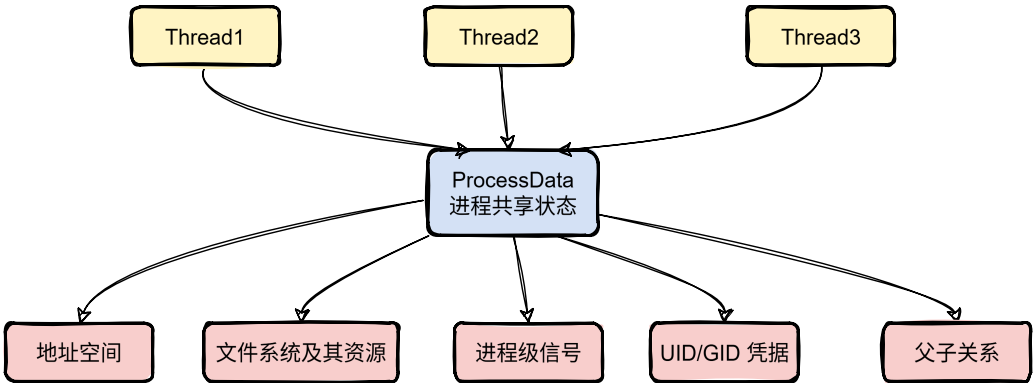


图　　与　　的双层对象模型

2.2.2

　　同时维护三类关系。

归属关系：多个线程通过共享 ProcessData 归属于同一进程。

亲缘关系：进程通过　　和　　形成父子树，供退出托管和　　使用。

分组关系：进程依次归入进程组和会话，形成　　、　　、　　三层标识。

　　这三类关系用途不同。线程组解决 哪些执行流共享资源 ，父子树解决 谁负责回收谁 ，进程组和会话解决 哪些进程作为一个作业被统一管理 。　　没有把它们压进同一张表，而是分别维护，再通过全局索引提供查询。

2.2.3

　　的进程身份设计参考了　　的凭据模型：　　不只保存一个值，而是拆成　　、　　、　　三个字段，　　采用相同结构。　　表示身份来源，　　用于当前权限判断，　　允许进程在受限范围内恢复先前身份。　　将　　三元组放入 ProcessData，让凭据真正参与程序执行、信号发送和　　权限检查。具体实现见 小节　　。

2.2.4

　　进程管理不是若干独立系统调用的集合，而是一条连续的生命周期：

创建 clone　　执行 exec　　退出 exit　　观察与回收 wait

　　创建阶段建立身份、资源和父子关系；　　替换程序映像，但保留　　、父子关系和进程组；

　　释放运行资源并留下退出状态；　　读取状态并完成最终回收。　　的多项修改都围绕这条闭环展开，尤其重视失败路径和中间状态，而不仅是系统调用成功时的主路径。

2.3

2.3.1　Thread　　ProcessData

　　Thread 和 ProcessData 是这部分最重要的两类数据，下面是关键代码片段。

```
pub struct Thread {
    // 共享进程级资源
    pub proc_data: Arc<ProcessData>,
    // 线程私有状态
    pub signal: Arc<ThreadSignalManager>,
    pub exit: Arc<AtomicBool>,
}
```

```rust
pub struct ProcessData {
    // 进程身份及组织关系
    pub proc: Arc<Process>,
    // 线程共享的进程资源
    pub aspace: Arc<Mutex<AddrSpace>>,
    pub scope: RwLock<Scope>,
    pub wait_status: SpinNoIrq<Option<ProcessWaitStatus>>,
    // UID/GID 三元组
    uid: AtomicU32, euid: AtomicU32, suid: AtomicU32,
    gid: AtomicU32, egid: AtomicU32, sgid: AtomicU32,
}
```

代码　　线程私有状态与进程共享状态

同一进程中的线程持有同一个 `Arc<ProcessData>`，因此能看到相同的地址空间、文件资源和凭据。`Thread` 自身则保存只与当前执行流相关的数据。　　新增字段时继续遵循这一边界，没有另建一套进程控制块。

2.3.2

原有退出码可以描述进程终止，但并不适合表示暂时停止或继续运行。　　增加了 `ProcessWaitStatus`：

```rust
pub enum ProcessWaitStatus {
    // 被信号暂停，但进程仍然存在
    Stopped(Signo),
    // 从停止状态恢复
    Continued(Signo),
}
```

代码　　进程停止与继续状态

退出状态仍由 `Process` 保存；停止和继续状态则独立记录在 `ProcessData` 中。图　　展示了进程在四个状态之间的完整流转关系：运行中与已停止之间通过　　互相切换，

触发进入僵尸状态，父进程调用　　后完成最终回收。父进程通过　　所能感知的三类事件（　　、　　、退出）都对应图中的状态转换。

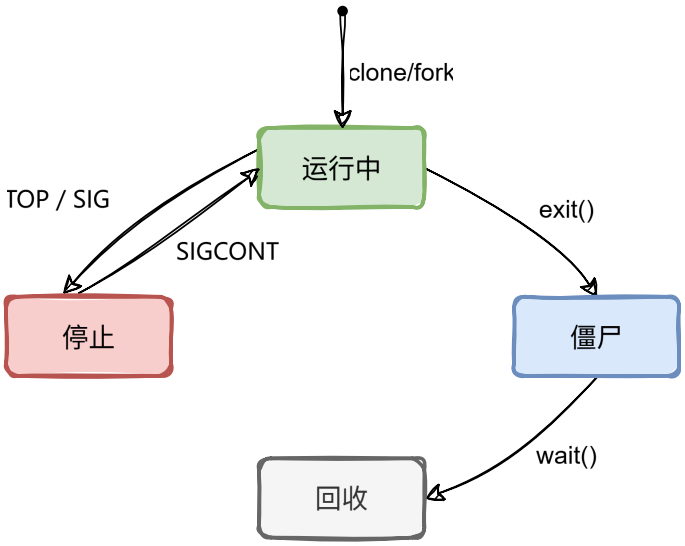


图　　进程生命周期状态机

2.3.3

系列调用的核心问题是：父进程怎样知道子进程发生了什么，以及知道之后是否立即消费这条状态。这个问题比表面上看起来复杂　　子进程可能停止、恢复，也可能退出，每种情况的处理语义完全不同。

以　　的作业控制为例。用户按下　　，前台进程组收到　　后暂停；此后执行　　或　　时，　　需要知道这些进程是 暂停等待继续 而不是 已经退出 。如果　　只能观测退出状态，　　就无法区分两者，作业控制便无从实现。　　正为此而来　　它让父进程感知子进程的停止状态，而不把子进程当作已终止。　　处理对称的情况：让暂停进程恢复运行，这次状态变化同样需要一个观测窗口才能被父进程感知。父进程可以等待指定　　、某个进程组或任意直接子进程，没有符合条件的直接子进程时返回 `ECHILD`。

选项	作用
`WNOHANG`	当前没有可报告状态时立即返回，不阻塞父进程
`WUNTRACED`	允许报告子进程停止状态
`WCONTINUED`	允许报告子进程继续运行状态
`WNOWAIT`	读取状态但暂不消费，后续仍可再次查询

表　　子进程等待的主要选项

如 图　　所示，停止、继续和退出都可以成为一次 `wait` 的报告。前两者只表示子进程的运行状态发生了变化：父进程读取报告后，子进程仍然存在，可以继续运行或再次被停止。退出状态则是最后一次报告；父进程按通常方式读取后，内核才回收这个僵尸进程。`WNOHANG` 用于当前没有可报告状态时立即返回，`WNOWAIT` 则允许父进程先查看结果、暂不消费，之后仍可再次读取。

传统的 `waitpid` 和 `wait4` 返回的是整数状态，调用者需要自己去解析，而　　新增的 `waitid` 使用 `siginfo_t` 结构化这些信息，能够区分正常退出、信号终止、停止和继续这些状态。　　将这些入口汇入同一套目标筛选和状态消费逻辑：不符合当前选项的状态会保留，等待之后合适的 `wait` 调用读取。

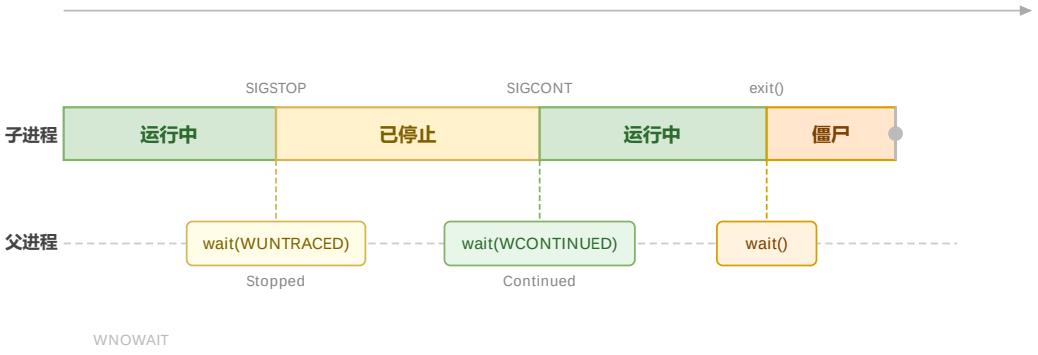


图　　子进程状态变化与父进程　　事件消费的时序关系

`waitid` 还可以通过　　选择目标。　　是指向进程对象的文件描述符，不会因为　　数值复用而转向另一个进程。配合 `WNOWAIT`，调用者可以先观察状态，之后再完成回收。父进程在暂时没有可报告状态时如何进入阻塞，以及状态变化怎样唤醒父进程，将在第三章结合通用等待机制说明。

2.3.4

内核需要按　　、　　、　　和　　查找对象。　　使用四张弱引用表分别索引线程、进程、进程组和会话：

索引表	键	作用
TASK_TABLE		查找具体线程和线程私有状态
PROCESS_TABLE		查找进程共享资源和父子关系
PROCESS_GROUP_TABLE		查找进程组并执行组级操作
SESSION_TABLE		查找会话及其进程组层级

表　　进程线程对象的全局索引

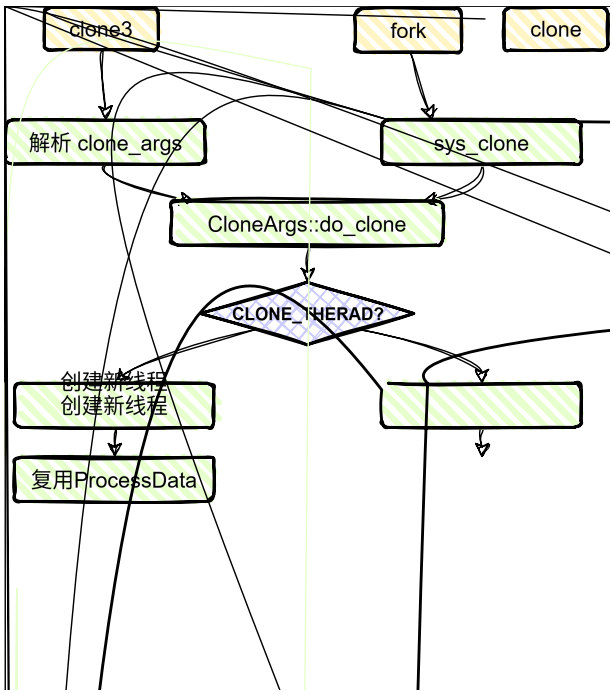
表中保存弱引用，不会因为全局索引而阻止对象释放。　　还修正了新建进程组和会话后的登记过程，避免对象已经创建，却无法通过　　或　　查到。

2.4

2.4.1 fork　　clone

fork 在　　中并未缺席。在　　平台上，sys_fork 会调用 sys_clone，其中不设置资源共享标志，只指定子进程退出时发送 SIGCHLD。因此，fork 创建的是拥有独立 ProcessData 的子进程；clone 和 clone3 则允许用户通过标志决定创建进程还是线程，以及资源如何继承。

这些入口最终都把参数整理为 CloneArgs，交给 CloneArgs::do_clone 执行。带有 CLONE_THREAD 时，新任务复用当前 ProcessData；否则调用 Process::fork 建立子进程及父子关系，再创建新的 ProcessData。地址空间、文件表、文件系统上下文和信号处理动作分别由 CLONE_VM、CLONE_FILES、CLONE_FS 和 CLONE_SIGHAND 决定共享或复制。



这些标志可以组合使用，但并非任意组合都合法。CLONE_THREAD 决定新任务是否仍属于原进程，其余标志主要控制资源如何继承，或指定创建前后的附加操作。

2.4.2

补充了 标志之间的依赖关系，让不合法的参数组合直接报错，而不是创建一个运行后才暴露问题的异常线程 进程，把 扼杀在摇篮里。例如，线程必须共享地址空间和信号处理动作，共享信号处理动作又必须共享地址空间：

```
// 线程必须共享地址空间和信号处理动作
if flags.contains(CloneFlags::THREAD)
    && !flags.contains(CloneFlags::VM | CloneFlags::SIGHAND)
{
    return Err(AxError::InvalidInput);
}
// 共享信号处理动作必须共享地址空间
if flags.contains(CloneFlags::SIGHAND)
    && !flags.contains(CloneFlags::VM)
{
    return Err(AxError::InvalidInput);
}
```

代码 标志组合校验

对于 clone3， 进一步检查结构体最小长度、栈地址与栈大小的配对关系，以及超出当前结构体版本的尾部字节。额外字节必须可读且全部为零，使旧内核能够安全接收新版本用户态传入的结构。

2.4.3

这里处理的情况和 的不同。 讨论的是子进程已经运行后，父进程怎样通过 wait 读取它的状态；如果子进程还没有开始运行，clone 就已经失败，那么系统调用会向调用者返回错误，但内核此前可能已登记 、父子关系和线程记录，必须把这些半成品全部撤销。

do_clone 会先建立进程对象和父子关系，分配 ProcessData，再创建线程记录并加入全局索引。若调用者请求 CLONE_PIDFD，内核还要分配文件描述符，并将其编号写回用户地址。只有这些步骤都完成后，新任务才会提交给调度器正式运行，如图 所示。

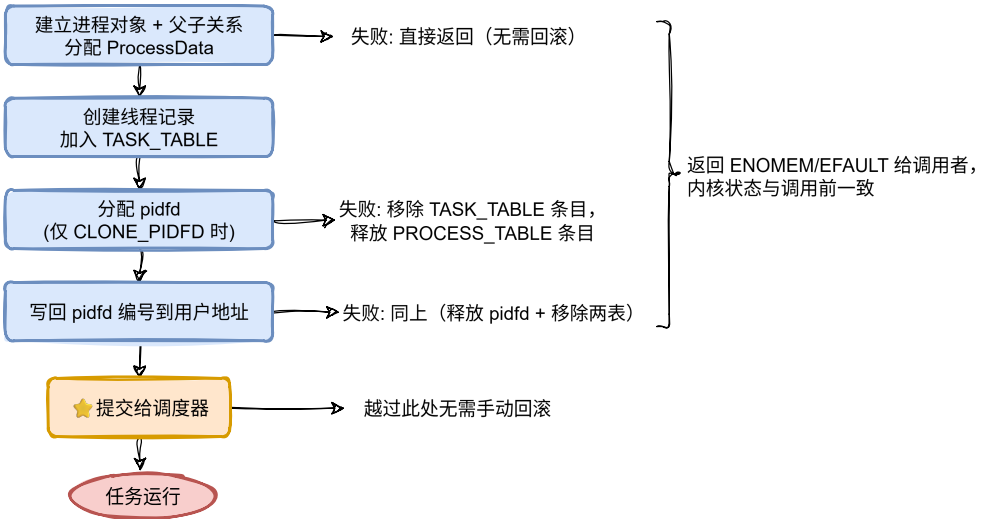


图 执行步骤与失败回滚路径

分配时文件描述符表可能已满，向用户地址写回　　编号时也可能遇到无效指针。这些错误发生时，进程对象已经登记进 PROCESS_TABLE，线程记录已经加入 TASK_TABLE，但新任务尚未运行，因此不会自行调用 exit() 进入僵尸状态。若直接返回错误，父进程会在 waitpid(-1, ...) 中等不到这个子进程的退出通知，　　也会被永久占用；PROCESS_TABLE 中的条目还会因父进程 children 表的强引用而无法自动释放。

因此在任务提交调度器之前统一处理失败路径：释放已分配的　　，从全局索引移除线程记录；若新进程没有其他存活线程，再一并释放进程对象。这样 do_clone 返回错误时，内核状态与调用前一致，调用者看到的 创建失败 才真正表示这次创建没有留下任何痕迹。

2.5

2.5.1 execve

的 execve 已经能够装载　　文件，替换地址空间和用户上下文，更新程序路径与命令行，重置信号处理动作，并关闭带有 CLOEXEC 标志的文件描述符。　　不创建新进程，因此　　、父子关系、进程组和会话保持不变。基线的不足在于只能按路径执行文件，对执行权限、MS_NOEXEC 挂载点和正在写入的可执行文件缺少检查，也没有 execveat。

2.5.2 execveat

传统 execve 只能接收路径，绝对路径依赖进程看到的文件系统，相对路径又依赖当前工作目录。程序若已经打开了可信目录或目标文件，仍需重新拼接并解析路径，期间目录项可能被改名或替换。execveat 可以从稳定的目录文件描述符开始解析，或使用 AT_EMPTY_PATH 执行已经打开的文件，减少重复路径解析带来的竞态，也便于容器运行时、动态加载器等程序在受控目录中启动目标。

增加该入口后，将 execve 与 execveat 汇入同一条 execve_common 路径，后续装载和状态更新仍由一套逻辑完成。在替换旧地址空间之前，　　先完成执行条件检查：

```
// 挂载点禁止执行
if exec_loc.mount_flags() & MS_NOEXEC != 0 {
    return Err(AxError::PermissionDenied);
}
// 按有效 UID/GID 检查执行位
if !has_exec_permission(exec_loc, proc_data.euid(), proc_data.egid())? {
    return Err(AxError::PermissionDenied);
}
// 拒绝正在被写入的可执行文件
if is_text_busy(exec_loc) {
    return Err(AxError::from(LinuxError::ETXTBSY));
}
```

代码　　程序执行前的约束检查

这些检查全部通过后，内核才装载新程序并提交地址空间。若解析、权限或装载失败，原程序仍可继续运行。成功后，内核还会清除旧程序遗留的线程退出指针，避免新地址空间继续引用失效地址。

2.6

2.6.1

一个进程可以包含多个线程，因此某个线程退出并不等于整个进程结束。内核先执行线程级退出清理，再把它从进程的线程集合中移除。清理内容包括　　和 `clear_child_tid`，其唤醒语义放在第三章的　　退出路径中说明。如果还有其他线程存活，共享的地址空间、文件表和 `ProcessData` 都继续保留。

只有最后一个线程退出时，内核才结束整个进程。此时进程进入僵尸状态，保存退出码，向父进程发送退出信号并唤醒正在等待的父进程。　　还在这一阶段释放文件锁、租约、共享内存记录和　　关系。进程对象不会立即删除，要等父进程调用 `wait` 读取退出状态后才完成回收。

2.6.2 exit_group

普通 `exit` 只结束调用线程，`exit_group` 则需要主动让进程中的全部线程退出。　　先标记线程组正在整体退出，再向其他存活线程发送终止信号。各线程仍按上一节的路径完成清理，最后离开的线程负责执行进程级退出工作。这一区别对多线程程序尤其重要。

2.6.3

进程结束后，还要处理它尚未退出的子进程。子进程不能随父进程一起删除　　它们之后退出时需要有进程负责读取退出状态，否则将永远停留在僵尸状态占用内核资源。　　沿用　　的做法，把这些子进程从原父进程的 `children` 表中移出，改由　　进程托管。如 图　　所示，整个过程只修改父进程指针，子进程本身的　　、　　和　　均保持不变，仍可继续运行。它们以后成为僵尸进程时，由　　调用 `wait` 完成回收。

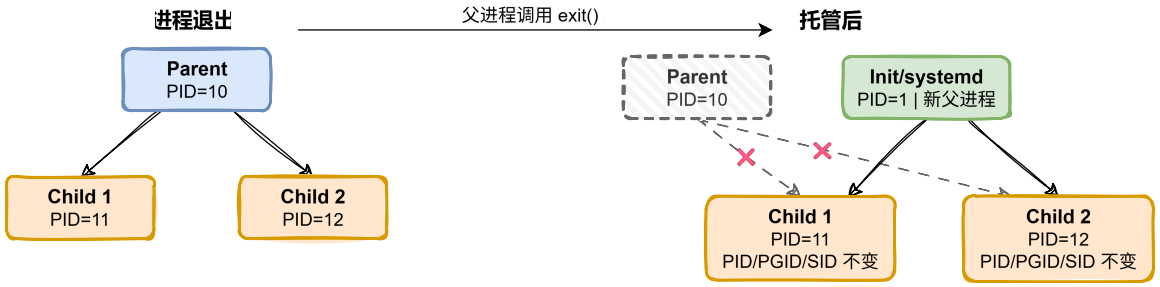


图　　孤儿进程托管示意图

2.7

2.7.1 PID　PGID　SID

进程组和会话用于组织多个相关进程。每个进程有　　，所属进程组有　　，进程组所属会话有　　。通常，组长的　　等于　　，会话首进程的　　等于　　。这种层级常见于　　：一条管道中的多个进程属于同一进程组，一个终端登录环境中的多个进程组属于同一会话。它们仍是独立进程，但可以按组查询、等待或发送信号。

在实现上，　　沿用　　的对象关系：`Process` 保存自身　　和所属的 `ProcessGroup`，`ProcessGroup` 保存　　、所属 `Session` 及组内进程，`Session` 再保存　　和会话内的进程组。

```
pub struct Process {
    pid: Pid,
    // 每个进程只属于一个进程组
    group: SpinNoIrq<Arc<ProcessGroup>>,
}
```

```
pub struct ProcessGroup {
    pgid: Pid,
    // 进程组归属于一个会话
    session: Arc<Session>,
    // 记录组内的所有进程
    processes: SpinNoIrq<WeakMap<Pid, Weak<Process>>>,
}

pub struct Session {
    sid: Pid,
    // 一个会话可以包含多个进程组
    process_groups: SpinNoIrq<WeakMap<Pid, Weak<ProcessGroup>>>,
}
```

代码　　进程、进程组与会话的对象关系

查询一个进程的三个标识时，内核沿对象引用逐层取得结果：先按　　找到进程对象，再沿所属关系取得进程组和会话标识。反向操作则依靠成员表完成　　向进程组发送信号时，内核遍历 ProcessGroup::processes，逐个找到组内进程。

2.7.2

已提供 setpgid、getpgid 等基础接口，但新建的进程组没有同步写入全局　　索引，导致其他系统调用可能无法通过　　找到该进程组。

根据目标　　分两种情况处理。目标　　等于进程自身　　时，内核创建新进程组并立即登记到全局索引，此后组信号和按进程组等待等操作都能查到该对象。目标　　指向已有进程组时，内核先从全局索引中找到目标组，检查它与当前进程是否属于同一会话，通过后将进程从旧组迁移到新组，同时更新全局索引。若目标组不存在或属于其他会话，则拒绝迁移。这样修改后，进程所属组、成员表和全局索引始终保持一致。

2.7.3

setsid 会同时创建新会话和新进程组。　　在创建成功后分别登记 Session 和 ProcessGroup：

```
if let Some((session, group)) = proc.create_session() {
    // setsid 同时创建会话和进程组
    add_session_to_table(&session);
    add_process_group_to_table(&group);
    // SID 等于调用进程的 PID
    Ok(session.sid() as _)
}
```

代码　　新会话与新进程组的创建和登记

2.8

2.8.1 UID/GID

进程的身份不能只用一个　　表示。系统既要知道它原本由谁启动，也要知道它此刻按谁的权限访问文件、执行程序或向其他进程发送信号。为此，　　沿用　　的三元凭据：

记录原始用户，　　用于当前权限检查，　　保存此前可恢复的有效身份；　　采用同样的结构。

这套设计用于受控的权限切换。普通用户运行带有　　属性的程序时，程序可以暂时以文件所有者的权限完成受保护操作，随后将　　降回普通用户。若后续仍需该权限，程序可从　　恢复，而不必始终以高权限运行。

基线中的部分身份设置接口只返回成功，并未保存身份变化；调用者看似完成了切换，后续权限检查却仍在使用旧身份。　　在 ProcessData 中加入完整的六个　　字段，并在 setresuid、setresgid 等接口中检查允许的转换范围：

```
let allowed = [old_ruid, old_euid, old_suid];
if old_euid != 0 {
    // 非特权进程不能把三元组改成从未拥有过的身份
    if !valid_id_change(ruid, &allowed)
        || !valid_id_change(euid, &allowed)
        || !valid_id_change(suid, &allowed)
    {
        return Err(eperm());
    }
}
// 分别更新三个字段，UID_GID_NO_CHANGE 表示不修改
if ruid != UID_GID_NO_CHANGE { proc_data.set_real_uid(ruid); }
if euid != UID_GID_NO_CHANGE { proc_data.set_euid(euid); }
if suid != UID_GID_NO_CHANGE { proc_data.set_suid(suid); }
```

代码　　三元组的保存与受限转换

普通进程不能凭空把自己改成另一个用户，只能在　、　　和　　三个已有值之间切换；　　为　的进程才可以设置新的身份。　　一旦改变，后续权限检查也随之改变：进程可能失去执行受保护文件的权限，不能向其他用户的进程发送 kill 信号，或不能通过　　附加调试目标。凭据由此成为实际的权限依据，而不是查询接口返回的一组固定数字。

　　则把传统上集中在　　身上的权限拆成多项能力。　　分别保存　　、　　、　　和　　集合：当前可使用的能力、允许启用的能力、可在执行新程序时传递的能力，以及能力的最高边界。更新这些集合时，内核会检查它们之间的约束，进程不能仅靠一次系统调用把原本没有的能力加入自身权限。

2.8.2　ptrace

　　新增　　的基础状态管理，建立 TRACEME、ATTACH、DETACH、停止、继续和　　报告之间的关系。　　时先检查目标是否存在、是否已被跟踪以及调用者是否具有相应权限；目标停止后，跟踪者通过　　获取事件。

但是当前实现只能完成最小控制流程。PTRACE_SYSCALL 和 PTRACE_SINGLESTEP 暂按普通继续处理，寄存器、目标内存和　　读写请求返回 EIO，因此还不能视为完整的　　后端。

2.9

　　选择沿用　　的进程线程框架，围绕四条主线完成加固：以 Thread 和 ProcessData 组成双层对象模型，以线程归属、父子树和进程组会话维护三类组织关系，以　、　、　构成三元凭据，并把　、　、　、　串成完整生命周期。

在此基础上，　　补严了 clone3 的　　和失败回滚，增加 execveat，完善子进程状态、等待接口和退出清理，并加入有状态的权限和　　控制。不足之处在于当前还未完成完整　　、多线程　　时仅保留调用线程的收敛、终端作业控制（前台进程组、后台读写限制和孤儿进程组规则）、　　隔离和　　数据访问，但进程从创建到回收的主要管理关系已经比基线更完整。

3

任务等待事件的过程看起来直接：检查条件、登记唤醒入口、进入阻塞，事件到了再恢复。但如果通知恰好在 条件检查完、 还没登记 这个窗口里到达时，等待方错过了这次通知，之后就没有人再来唤醒它了。本章大多数改动都对准这个竞争窗口，以及由此派生出的几个具体问题：超时后旧等待项还留在队列里，子进程状态变化和父进程登记 之间有空隙，通知与入队之间存在同样的竞争。

沿用 的 调度框架，改动集中在进入阻塞和重新唤醒之间的那段路径。任务在就绪、运行和阻塞之间循环，执行完毕后退出调度流程，整体关系如图所示。

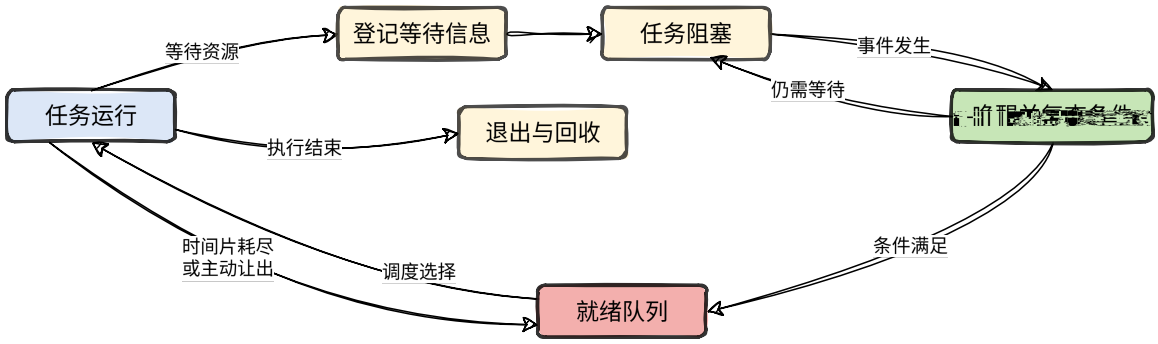


图 调度、阻塞与唤醒的总体循环

3.1

已经提供了可运行的任务调度骨架：axtask 管理任务状态和运行队列，axsched 用 调度器按时间片轮转，多核下每个 有独立就绪队列， 的 、位掩码唤醒、重新排队和 也已实现，定时器与子进程等待接口也在。这部分 主要是在 的基础上进行补充的。

问题集中在事件交错时的状态处理上。 等待项只保存了 和位掩码，没有独立的通知状态，超时或被信号中断后旧记录还留在队列里 内核说不清通知究竟有没有到达，失效等待项也会占用后续唤醒名额。FUTEX_WAIT_BITSET 的绝对截止时间被当相对时长用，定时器重新设置后旧事件照样留在全局队列。子进程状态变化与父进程登记 之间，也同样有一段通知可能悄悄溜走的窗口。

保留 调度框架，改动集中在三个环节：等待者如何登记，通知如何与阻塞状态配合，等待结束后如何清理记录。对应关系如下：

内容	基线	的增量
任务调度	任务状态、 时间片和 亲和性	沿用调度框架，修正亲和性位图解析
阻塞唤醒	、 和等待队列	调整 唤醒过程，补充条件复查和等待项清理
定时处理	基础休眠和进程定时器	区分相对与绝对时间，使用 过滤旧事件
	、 、位掩码和重新排队	加入独立通知状态，修正超时语义和退出清理
子进程等待	退出事件与基础等待	将状态变化接入 ，并在登记后复查条件

表 调度与同步机制的主要增量

这些修改共同作用于任务离开　　和重新获得　　的过程。其中最难处理的是通知与状态转换可能同时发生：任务检查完条件、尚未登记　　时，事件可能已经到达；任务准备进入阻塞时，通知也可能先一步发生。下面的叙述会先用任务状态、抢占和上下文切换说明调度基础，再围绕这一竞争窗口讨论通用阻塞唤醒机制，最后分析定时等待、子进程等待和　　的具体处理。

3.2

3.2.1

任务调度围绕四种状态展开。调度器不需要理解任务正在等待哪一种资源，只需判断任务当前属于以下四种中的哪种状态：

```rust
pub enum TaskState {
    Running = 1, // 正在 CPU 上执行
    Ready   = 2, // 可以运行，正在队列中等待
    Blocked = 3, // 正在等待事件，暂时不能运行
    Exited  = 4, // 已经结束，等待资源回收
}

pub struct TaskInner {
    // 调度器通过该字段判断任务当前能否运行
    state: AtomicU8,
    // ...
}
```

代码　　任务状态的数据结构

`state` 使用原子整数保存任务状态。定时等待、　　和子进程等待各自维护不同的业务条件，但要重新获得　　，都必须让任务从 `Blocked` 回到 `Ready`。调度器只处理这一状态转换，并不判断具体等待条件是否已经满足。四种状态之间的转换关系如图所示。

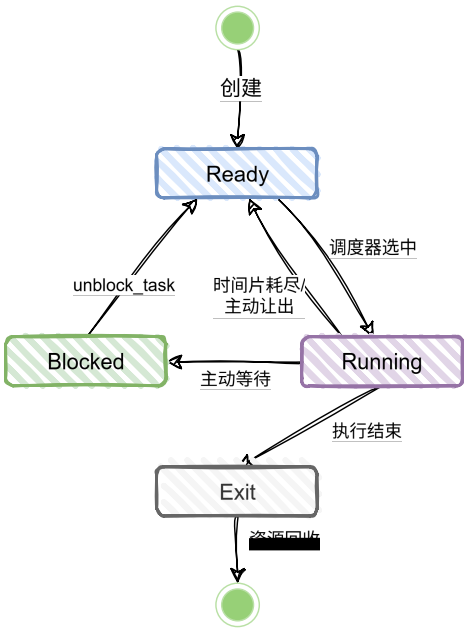


图　　的状态转换关系

新任务先以 `Ready` 状态进入就绪队列，被调度器选中后变为 `Running`。主动让出　　的任务仍具备运行条件，因此会回到 `Ready`；等待资源的任务则进入 `Blocked`，由 `unblock_task` 恢复

为 Ready。任务执行结束后进入 Exited，并在资源回收后离开状态机。　　的定时等待、　　和子进程等待都通过这组状态转换重新接入调度器。

状态修改通过原子状态和运行队列中的唤醒检查配合完成。多个通知同时到达时，唤醒路径应该只允许仍处于 Blocked 的任务重新进入就绪队列。

3.2.2

任务按照　　策略轮流使用　　。多核环境下，每个　　都有自己的就绪队列，用来保存处于 Ready 状态的任务。调度器从队头选择下一个任务，任务的默认时间片是　个时钟滴答；时间片用完后，任务回到队尾，　　转交给后面的任务。

时钟中断每到来一次，scheduler_timer_tick（调度器的时钟节拍处理函数）就更新当前任务的时间片。计数归零时，内核先设置待抢占标志，而不是立刻在中断处理或持锁位置切换任务。等代码离开禁止抢占的临界区后，再执行真正的重新调度。

时钟中断	时间片减一	设置待抢占标志	离开临界区	重新调度

时间片归零后，内核先记录抢占请求，等执行离开禁止抢占的临界区再调度。任务如果在修改运行队列、处理中断或持有关键锁时被换出，其他　　可能看到尚未完成的中间状态。延迟抢占把上下文切换放到状态一致的位置，运行队列和锁的处理也能先行结束。

如果当前没有普通任务可运行，　　会进入　　任务（没有普通任务时运行的空闲任务），等待新的任务或中断到来。

3.2.3

前两节已经说明了任务状态如何变化，以及时间片耗尽后如何发起抢占。本节从重新调度真正开始的位置继续：当前任务的状态处理完毕后，resched 从本　　的就绪队列中选出下一个任务；如果队列为空，就让　　任务暂时接管　　。

```
fn resched(&mut self) {
    // 从就绪队列选择任务；队列为空时运行 idle 任务
    let next = self.scheduler.lock().pick_next_task()
        .unwrap_or_else(|| unsafe {
            IDLE_TASK.current_ref_raw().get_unchecked().clone()
        });

    self.switch_to(crate::current(), next);
}

fn switch_to(&mut self, prev: CurrentTask, next: AxTaskRef) {
    // 将选中的任务设为正在运行
    next.set_state(TaskState::Running);

    // 保存当前任务现场，恢复下一个任务现场
    let prev_ctx = prev.ctx_mut_ptr();
    let next_ctx = next.ctx_mut_ptr();
    unsafe { (*prev_ctx).switch_to(&*next_ctx) };
}
```

代码　　从选择任务到切换现场的主要流程

选出任务后，switch_to 先将它标记为 Running，再切换任务现场。这里的 现场 是任务暂停前的运行位置。每个任务都有一个 TaskContext，其中保存内核栈指针、需要保留的寄存器和返回位置。切换时，当前 的这些值被保存到旧任务中，再从新任务的 TaskContext 恢复。旧任务以后再次被选中时，就能接着原来的位置运行。

如 图 所示，两个任务的 TaskContext 分别位于各自的内核栈中，switch_to 把 的寄存器值写出，再从 读入；其中 satp 让切换完成的瞬间就生效了新的地址空间。

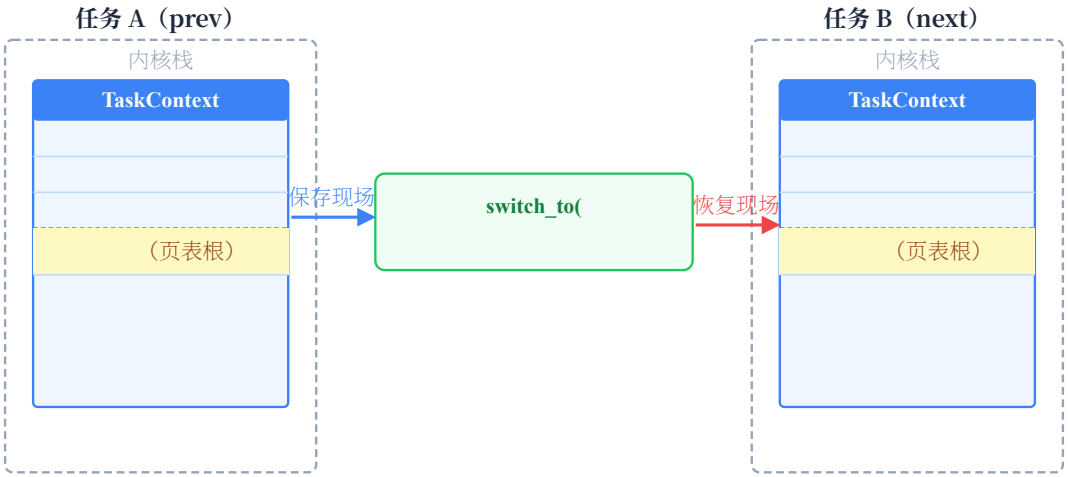


图 布局与 的现场传递方向

对于用户任务， 在创建任务时把该进程的页表根写入 TaskContext，保证切换后使用正确的地址空间。用户态寄存器则保存在 UserContext（用户程序暂停时的寄存器记录）中。至此，从 的抢占请求到完成切换，可以连成下面这条流程：

设置待抢占标志 到达安全调度位置 选择下一任务 保存旧现场 恢复新现场

3.2.4 CPU

每个任务都保存一份 亲和性掩码 cpumask，用来限制它可以在哪些 上运行。例如，掩码只包含 和 时，任务就不会进入其他 的就绪队列。

新任务默认可以使用所有已经启用的 。任务创建或被唤醒时，调度器在 cpumask 允许的范围内轮转选择一个 ，再把任务加入该 的就绪队列。

```
fn select_run_queue_index(cpumask: AxCpuMask) -> usize {
    loop {
        // 在所有 CPU 编号之间轮转
        let index = RUN_QUEUE_INDEX.fetch_add(1, Ordering::SeqCst)
            % axconfig::plat::CPU_NUM;
        // 只选择亲和性掩码允许的 CPU
        if cpumask.get(index) {
            return index;
        }
    }
}
```

代码 在亲和性范围内选择

这段逻辑按 编号轮转，并跳过亲和性掩码不允许的 。它沿用原有的多运行队列设计， 在这一部分的实际修改主要位于用户位图解析和设置后的任务迁移。

用户传入的亲和性范围是一张位图，每一位对应一个　　。　　修正了这张位图的解析过程：内核按照实际配置的　　数量读取内容，再把其中被置为　的位置写入任务掩码。用户程序通过　　的 sched_getaffinity 查询范围，通过 sched_setaffinity 修改范围。

```
// 用户位图中的每一位对应一个 CPU
for cpu_id in 0..axconfig::plat::CPU_NUM {
    if user_mask[cpu_id / 8] & (1 << (cpu_id % 8)) != 0 {
        cpu_mask.set(cpu_id, true);
    }
}

// 保存新的亲和性范围
let curr = current().clone();
curr.set_cpumask(cpu_mask);

// 当前 CPU 已不在范围内时，将任务迁移到允许的 CPU
if !cpu_mask.get(axhal::percpu::this_cpu_id()) {
    let migration_task = TaskInner::new(
        move || migrate_entry(curr),
        "migration-task".into(),
        4096,
    ).into_arc();
    current_run_queue().migrate_current(migration_task);
}
```

代码　　亲和性掩码的设置与任务迁移

新的掩码保存后还要检查任务当前所在的　　。如果该　　仍被允许，任务可以继续运行；如果已经被排除，　　会把任务迁移到另一个允许的运行队列。新的设置因此会立即改变任务实际运行的位置，而不只是留下一个供查询的数值。

用户调用 sched_yield 主动让出　　后，　　将当前任务放回就绪队列，再由调度器选择其他任务运行。这个接口适合任务暂时没有紧急工作、希望让其他就绪任务先执行的情况。

3.3

　　延续　　的　　模型。这套机制的实际好处在于：　　、　　、　　等来源各不相同的等待，都能挂到同一套唤醒路径上，block_on 也因此能以统一方式暂停和恢复任务，不需要为每类等待单独实现阻塞逻辑。代价是每个等待路径都得正确维护通知状态和双重检查，否则竞争窗口就会漏掉通知。这正是本节要讨论的。

3.3.1

调度器只知道任务处于 Blocked，并不知道任务在等什么。事件对象和等待者之间还需要一套通知协议。　　沿用　　表示 尚未完成、之后还要继续检查的操作 ，用　　记录 条件可能变化时如何通知当前任务 。

以读取管道为例，任务先检查缓冲区。已有数据就直接读取；没有数据则把自己的　　登记到管道中，再由　　返回 暂时未完成 。另一个任务写入数据后，管道调用保存的　　，通知读任务继续检查。



图 事件登记与通知的基本流程

关键步骤是登记 后的第二次条件检查。假如数据恰好在第一次检查结束后到达，此时写入方还找不到等待者，无法调用 。登记完成后立刻复查缓冲区，读任务仍能看到已经到达的数据，不会因为漏掉那次通知而一直等待。

收到通知后还要重新检查条件。 只表示 情况可能有变化 ，并不保证资源一定属于当前任务。例如多个读任务同时被通知时，管道中的数据可能已经被其中一个任务取走，其他任务仍需继续等待。

3.3.2 Waker PollSet

是 留给事件对象的任务唤醒入口。事件对象不必直接操作运行队列，只需保存等待者提供的 ，并在状态发生变化时调用 `wake()`。 继续使用这一接口，把新增的 、 和 等等待路径接到同一套任务唤醒流程上。

`PollSet` 管理一组 ，管道、 、 、 和终端等对象都通过它登记等待者。

增加 `has_waiters`，没有等待者时直接返回；真正唤醒时，先在锁内取出并清空本轮 ，再到锁外逐个调用。内部锁只保护集合操作，不覆盖可能继续触发调度的 `wake()`。

```
pub struct PollSet {
    inner: Lazy<Mutex<Inner>>,
    // 没有等待者时，wake 可以直接返回
    has_waiters: AtomicBool,
}

pub fn wake(&self) -> usize {
    if !self.has_waiters.load(Ordering::Acquire) {
        return 0;
    }
    // 取出并清空本轮登记的 Waker
    let (wakers, len) = self.inner.lock().take_wakers();
    self.has_waiters.store(false, Ordering::Release);
    // 真正唤醒时已经不再持有内部锁
    for waker in wakers.iter().take(len) {
        unsafe { waker.assume_init_read() }.wake();
    }
    len
}
```

代码 取出并唤醒等待者

`take_wakers` 在取出 的同时清空集合，因此后续事件不会再次使用这一批旧记录。

只负责 有人需要重新检查了 ，不会替业务对象判断条件是否真的满足。管道仍要检查缓冲区，子进程等待仍要检查状态， 也要检查通知标记。

3.3.3 block_on

通过反复轮询判断操作是否完成，而同步系统调用需要在结果尚未产生时暂停当前任务。block_on 连接这两种执行方式： 返回 Ready 时，系统调用取得结果；返回 Pending 时，当前任务准备阻塞，把 交给其他任务。任务选择和现场切换仍由 的调度路径完成。

```
fn wake_by_ref(self: &Arc<Self>) {
    if let Some(task) = self.task.upgrade() {
        // 先记录通知已经到达
        *self.woke.lock() = true;
        // 再把阻塞任务放回合适的运行队列
        select_run_queue(&task).unblock_task(task, false);
    }
}

match future.poll(&mut context) {
    // 没收到通知，当前任务可以安全进入 Blocked
    Poll::Pending if !*woke => run_queue.blocked_resched(woke),
    // 通知先到，不再阻塞，只让其他任务先运行一次
    Poll::Pending => yield_now(),
    Poll::Ready(output) => return output,
}
```

代码 与 的阻塞唤醒配合

返回 Pending 与通知到达可能发生在相邻时刻。 先把 woke 设为真，再尝试唤醒任务；阻塞路径则在改变任务状态前检查这个标记。通知先到时，任务不再进入 Blocked，只让出一次 后重新轮询。阻塞先完成时，unblock_task 再把任务放回就绪队列。woke 记录 本轮轮询后是否已经收到通知 ，补上了 状态与任务状态之间的空隙。

图 对比了这两条路径的时序差异。

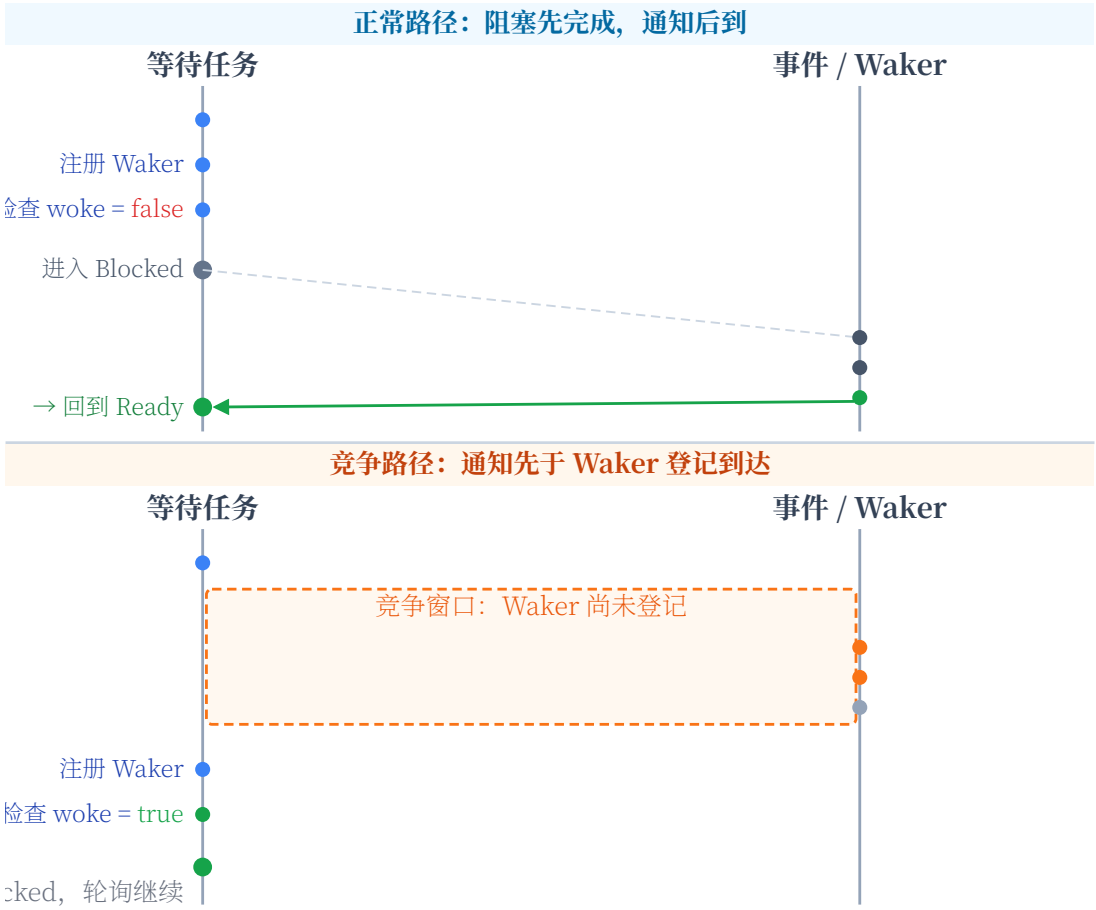


图　　正常阻塞路径与　　标记覆盖竞争路径的时序对比

正常事件之外，信号也可以结束等待。`interruptible` 同时检查原来的　　和任务的信号状态。收到未屏蔽信号后，等待提前结束，上层系统调用再决定返回 `EINTR`、保存剩余时间或按信号规则重启。

等待因超时或信号提前结束后，还必须删除本轮登记。否则后续事件仍可能找到旧　　，既产生无效唤醒，也会干扰唤醒数量统计。　　会清空已经取出的　　，　　则按等待项精确清理，具体过程将在　　节说明。

3.4

　　节建立了 检查条件、登记　　、阻塞、收到通知后复查 的通用流程。本节选择两类应用说明这套流程如何落地。定时等待关注截止时间是否到达，子进程等待关注是否出现可报告的状态。二者的业务条件不同，但都不能把一次唤醒直接当作最终结果，任务恢复后仍要重新检查条件。

3.4.1

定时等待先根据系统调用参数选择时间来源，再把等待目标统一换算为截止时间。单调时钟只向前推进，适合表示 再等待多久 ；实时时钟对应系统日历时间，适合指定一个实际时刻。

时间参数分为相对时长和绝对时间。相对时长需要加到当前时间上，绝对时间本身就是截止时间；如果截止时间已经到达，等待立即结束。统一转换后，后续逻辑只需比较 当前时间是否达到截止时间 。同样的区分也用于修正后文 `FUTEX_WAIT_BITSET` 的超时语义。

`nanosleep` 使用单调时钟完成相对休眠。`clock_nanosleep` 还允许选择时钟，并通过 `TIMER_ABSTIME` 表示绝对时间。休眠期间如果收到未屏蔽信号，　　会结束本次等待，向用户返回 `EINTR`，并写回尚未用完的时间。

3.4.2

的定时事件可以按截止时间触发，但定时器重新设置后，旧事件可能仍留在队列中。仅凭截止时间无法判断取出的条目是否仍然有效。　　改用 `BTreeMap` 保存定时器，并用截止时间 `deadline` 和递增序号 `seq` 组成唯一的 `TimerKey`。后台定时任务每次取出最早到期的条目，到期后更新定时器状态并发送相应信号。

```
struct TimerKey {
    // 定时事件应当触发的时刻
    deadline: Duration,
    // 区分截止时间相同的多个事件
    seq: u64,
}

struct AlarmRuntime {
    // 为新事件生成不重复的序号
    next_seq: u64,
    // 按到期顺序保存全部定时事件
    timers: BTreeMap<TimerKey, Entry>,
}
```

代码　　定时器队列的关键数据结构

同一时刻可能有多个定时器到期，`seq` 使每个事件都有独立的键。重新设置或取消定时器时，内核删除旧键；后台任务取出事件后，还要确认它与定时器当前保存的键一致。只有匹配的事件才会触发信号，重新设置前留下的旧条目会被忽略。新增更早的定时器时，系统还会通知后台任务重新计算下一次等待时间。

每个线程还有一个时间管理对象 `TimeManager`，用来分别累计用户态和内核态运行时间。实际时间定时器 `ITIMER_REAL` 按经过时间发送 `SIGALRM`；用户时间定时器 `ITIMER_VIRTUAL` 只统计用户态时间；性能分析定时器 `ITIMER_PROF` 同时统计用户态与内核态时间。周期定时器到期后重新装载，一次性定时器到期后停止。

3.4.3

第二章已经介绍了子进程状态、等待目标和接口语义。本节只讨论等待过程如何接入任务阻塞与唤醒。父进程暂时找不到可报告状态时，需要登记通知入口并暂停运行；子进程状态变化后，再将父进程唤醒。

父进程调用　　时先筛选目标子进程，再检查是否已有可报告状态。如果暂时没有结果，它就把　　登记到 `child_exit_event` 并进入等待。子进程退出或状态发生变化时会触发这个事件，父进程恢复运行后重新读取状态。

```
// 先检查是否已有子进程状态可以报告
match check_children().transpose() {
    Some(result) => Poll::Ready(result),
    None => {
        // 暂无结果，登记父进程的 Waker
        child_exit_event.register(cx.waker());
        // 登记后复查，覆盖同时发生的状态变化
        match check_children().transpose() {
```

```
                Some(result) => Poll::Ready(result),
                None => Poll::Pending,
            }
        }
    }
```

代码　　父进程等待子进程状态的实现

父进程因此不必循环查询子进程。状态变化触发　　，父进程恢复后重新执行 check_children，再根据第二章所述的目标和选项判断是否已有结果。代码中的登记后复查与　　的通用流程一致，用来覆盖 首次检查结束、　　尚未登记 期间发生的子进程状态变化。

3.5 Futex

3.5.1

把无竞争的同步留在用户态。线程先通过原子变量尝试获得锁，只有发现资源被占用时才进入内核等待。因此，大多数没有竞争的加锁和解锁不需要系统调用。

进入内核后，　　使用 FutexKey 定位等待队列。进程私有　　以用户地址为键；共享匿名映射和文件映射还要结合共享后端与区域偏移，保证不同进程即使把同一页映射到不同虚拟地址，也能找到同一个　　。

每个　　表项保存等待队列和 owner_dead 状态。为了分清正常唤醒、超时和信号中断，　　将单个等待项组织为：

```
pub struct WaitQueue {
    // 每个等待项依次保存:
    // 通知状态、唤醒位掩码和任务唤醒入口
    queue: SpinNoIrq<
        VecDeque<(Arc<AtomicBool>, u32, Arc<PollSet>)>
    >,
}
```

代码　　等待队列的数据结构

元组中的 AtomicBool 是 notified，记录等待项是否已经被选中唤醒；u32 保存　　；PollSet 保存任务的唤醒入口。业务状态与唤醒动作由此分开：notified 防止通知丢失或重复处理，　　负责让任务重新参与调度。

图　展示了　　如何定位　　，以及每个等待项三个字段的分工　notified 记录通知结果，　　过滤唤醒目标，　　持有唤醒入口。



图　　、　　与等待项三元组的结构关系

3.5.2 WAIT WAKE

等待者只有在用户地址仍等于期望值时才应睡眠。系统调用入口先做一次快速检查，不相等就返回 EAGAIN；准备入队时，内核在持有等待队列锁的情况下再次读取用户值。第二次检查把用户值判断与入队操作放进同一个受保护区间，条件已经改变时，本次调用不再阻塞。

条件仍成立时，内核创建等待项并进入可中断、可超时的 。FUTEX_WAKE 从队列中选择指定数量的等待者，先把 notified 设为真，再调用 ；被正常唤醒的等待项同时从队列删除，不会继续占用下一次 的名额。

与 不同，这里额外处理 等待项入队后的竞争：notified 由 FUTEX_WAKE 写入，并在登记 前后复查。它记录的是单个 等待项的通知结果，后续位掩码唤醒、超时清理和重新排队都会保留或处理该状态。

3.5.3

位掩码唤醒与重新排队也使用新增的 notified 状态和退出清理。FUTEX_WAIT_BITSET 为等待者附带位掩码，FUTEX_WAKE_BITSET 只唤醒 与目标 有交集的任务，从而避免叫醒无关线程。

FUTEX_REQUEUE 先唤醒一部分任务，再把其余等待项移动到另一个 队列。FUTEX_CMP_REQUEUE 在此之前还会比较原地址的用户值，不匹配时返回 EAGAIN。移动过程中保留 notified、 和 ，不重新登记 。

这类操作常用于 条件变量。广播时不必把所有线程同时叫醒争抢互斥量，可以先唤醒少量线程，再把其他等待者转移到互斥量对应的队列。如 图 所示，操作结束后原队列只剩余未被处理的等待者，其余项带着原有的 notified、 和 整体迁移到目标 队列。

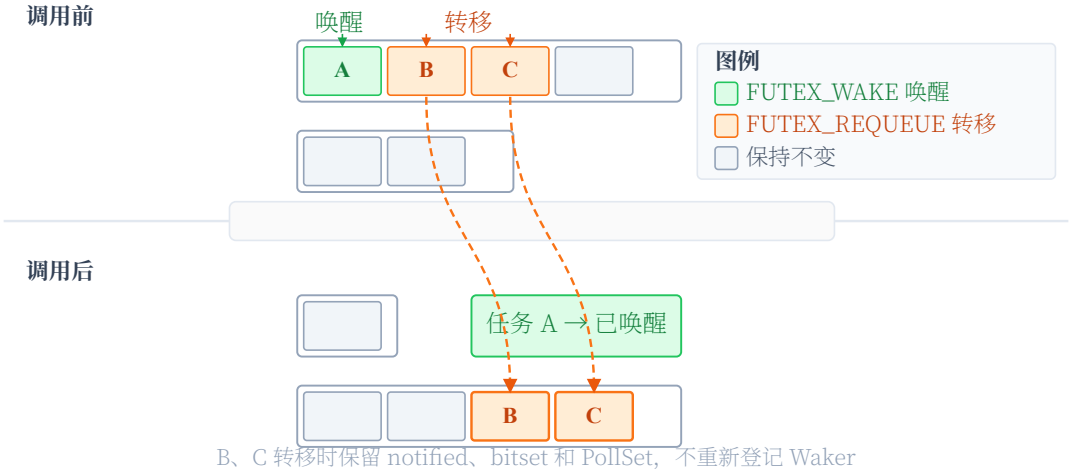


图 将部分等待者唤醒、其余等待者转移至另一 队列

3.5.4

等待可能正常收到通知，也可能先超时或被信号打断。 用 interruptible(timeout(...)) 组合这三条路径。超时返回 ETIMEDOUT，信号中断返回 EINTR；两种提前结束都会按照 notified 的对象地址精确删除当前等待项。

原有实现没有把 FUTEX_WAIT 和 FUTEX_WAIT_BITSET 的时间语义分开。 按 规则处理：前者接收相对时长，后者接收绝对截止时间，并根据 FUTEX_CLOCK_REALTIME 选择实时时钟或单调时钟。

独立通知状态解决了 通知是否已经到达 的判断问题，前后复查覆盖登记窗口，退出清理则删除已经失效的等待项。三者分别处理通知丢失、重复处理和队列残留。相关修改修复了 上 futex_wait、 和部分休眠测试卡住的问题。

3.5.5 robust list

线程持有互斥量时如果直接退出，其他线程可能永远等不到解锁，从而造成死锁。针对这种情况， 完善了退出时的 清理。用户态线程库通过 set_robust_list 把 头指针交给内核，链表记录该线程可能持有的 。

线程退出时， 按 futex_offset 找到对应 ，将 owner_dead 设为真，并唤醒一个等待者。被唤醒线程收到 EOWNERDEAD 后，由用户态线程库检查共享数据并恢复锁的一致性。链表遍历受 ROBUST_LIST_LIMIT 限制，损坏的用户链表不会让内核无限循环。

线程退出还会清零 clear_child_tid，并唤醒这个地址对应的 。这条路径支持 pthread_join 等线程回收操作。

3.6

已经具备 调度、任务状态转换、上下文切换和基础事件等待， 没有另建一套调度器。本章的修改集中在调度器之外、重新进入调度器之前的那段路径：等待者登记状态，事件到来后留下通知，任务恢复时复查条件，等待提前结束时清理旧记录。

缩短了持锁唤醒路径，block_on 用 woke 协调通知与阻塞，定时器用 TimerKey 识别仍然有效的事件，子进程等待在登记 后复查状态， 则通过 notified、双重检查和精确删除处理竞争。它们最终回到同一条流程：条件不满足时阻塞，条件可能变化时唤醒，复查成功后转为就绪，再由调度器安排运行。

4

4.1

已经具备用户虚拟内存管理的主体框架，支持程序装载、用户堆栈、动态映射、缺页处理和写时复制。 沿用这一设计，重点补全映射语义、权限管理和缺页处理的细节，并优化频繁 fork 场景下的 路径。

整体上，系统用分页机制隔离进程，把连续的虚拟地址按页映射到离散的物理内存。程序装载、堆栈和动态映射先确定可用地址范围与访问权限，物理页推迟到首次访问时分配；发生缺页后，内核根据区域用途从匿名内存、文件缓存、共享内存或固定物理地址取得页面。进程复制通过写时复制延后私有页面的实际拷贝。映射解除或进程退出时，内核依据页面所有权和引用关系决定是否写回脏数据、是否释放物理页。

4.2

内存管理需要同时处理三件事：哪段虚拟地址可以怎样访问、物理页从哪里来、当前已经装入了哪些页面。把它们混在一个结构里，fork 时就不得不为每种页面来源写单独的复制逻辑，mprotect 也要同时操作区域描述和页表。 把三者分开：MemoryArea 只记录区域范围和权限，Backend 只负责填页和释放，PageTable 只保存已装入的快照。三者职责不交叉，各操作各自只改自己负责的部分。

代码 展示了三者的包含关系。

```
AddrSpace
├── va_range              可用虚拟地址上下界
├── areas: MemorySet
│   └── BTreeMap<VirtAddr, MemoryArea>
│       ├── va_range      连续的虚拟地址范围
│       ├── flags         读 / 写 / 执行 / 用户态权限
│       └── backend       Linear | Cow | Shared | File
└── pt: PageTable         已装入的虚拟页 → 物理页快照
```

代码 三层结构概览

4.2.1 AddrSpace MemoryArea

每个进程通过 ProcessData 持有一个 Arc<Mutex<AddrSpace>>。同一进程内的线程共享该对象，看到相同的代码段、数据段、堆和共享映射；普通 fork 创建新的 AddrSpace，带有 CLONE_VM 的 则继续共享原地址空间。

AddrSpace 的核心字段：

```rust
pub struct AddrSpace {
    va_range: VirtAddrRange,    // 允许用户进程使用的虚拟地址范围
    areas: MemorySet<Backend>, // 按起始地址组织的虚拟内存区域
    pt: PageTable,              // 已生效的虚拟页到物理页映射
}
```

代码 用户地址空间的核心数据结构

MemorySet 是一组互不重叠的 MemoryArea，以区域起始地址为键存在 BTreeMap 里，便于查找相邻区域和检测重叠。每个 MemoryArea 保存虚拟地址范围、访问权限和 Backend：

```rust
pub struct MemoryArea<B: MappingBackend> {
    va_range: AddrRange<B::Addr>, // 一段连续的虚拟地址范围
    flags: B::Flags,              // 读、写、执行等权限
    backend: B,                   // 决定页面从何处取得
}
```

代码 的核心字段

区域存在不等于页面已经分配。程序可以拥有一个数百页的匿名区域，而页表中只有实际访问过的几页。这样既保留了连续的虚拟地址范围，也避免为未使用页面占用物理内存。

地址空间操作均围绕这三个字段展开：`map` 增加区域，`unmap` 删除区域和页表项，`protect` 修改权限，`populate_area` 为指定范围准备页面，`handle_page_fault` 处理访问过程中出现的缺页。

4.2.2 Backend

把不同来源的页面封装为统一接口。`AddrSpace` 不需要了解文件读取、共享页引用和 复制的细节，只在映射、填页、解除映射和复制地址空间时调用 。

```rust
#[enum_dispatch(BackendOps)]
pub enum Backend {
    Linear(LinearBackend), // 按固定偏移建立虚拟地址到物理地址的映射
    Cow(CowBackend),       // 管理匿名页和私有映射，fork 后执行写时复制
    Shared(SharedBackend), // 让多个地址空间映射同一组物理页
    File(FileBackend),     // 将文件缓存页映射到用户地址空间
}
```

代码 的四种类型

接口中的 `populate` 负责让指定范围满足本次访问权限，`clone_map` 负责地址空间复制。

4.3

4.3.1

页表把虚拟页号映射到物理页号，并保存读、写、执行和用户态等权限位。 支持 和 两种体系结构，各自使用不同的多级页表格式，但通过类型参数共享同一套通用实现。

以 为例， 位有效虚拟地址按如下位域划分：

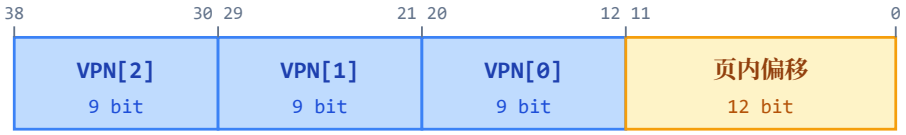


图 虚拟地址位域划分

三级页表从 寄存器指向的根页表出发， 索引第一级， 索引第二级， 索引叶子项，最终得到物理页号。 采用四级页表，结构原理相同，仅级数与位域宽度不同。

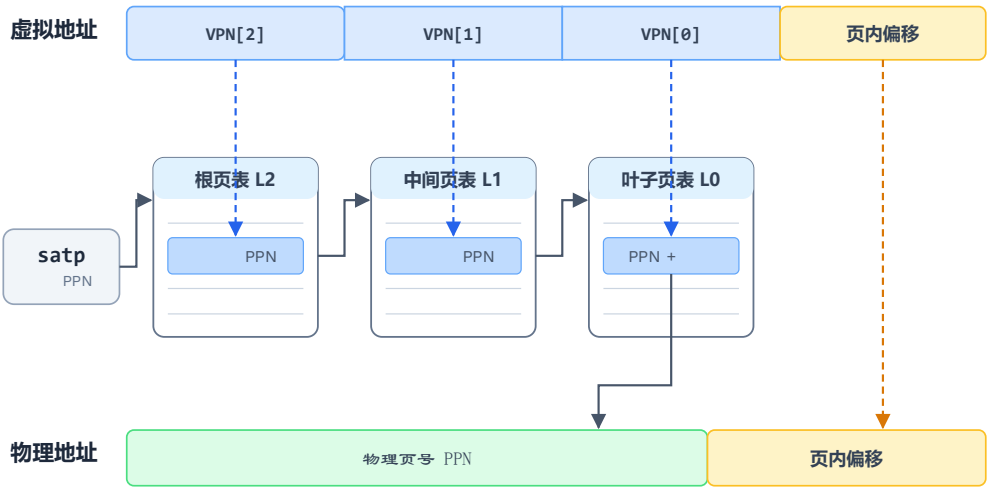


图　　三级页表查找过程（　　叶子页）

axhal 根据编译目标导出对应的 PageTable 类型，上层统一通过 PageTableMut 接口操作，不感知体系结构差异：

```
#[cfg(target_arch = "riscv64")]
pub type PageTable = Sv39PageTable<PagingHandlerImpl>;

#[cfg(target_arch = "loongarch64")]
pub type PageTable = LA64PageTable<PagingHandlerImpl>;

pub struct PageTable64<M, PTE, H> {
    root_paddr: PhysAddr,        // 根页表的物理地址
    _phantom: PhantomData<(M, PTE, H)>,
}
```

代码　　跨体系结构页表的类型组织

页表只保存 当前这一页映射到哪里、允许什么访问 ，文件路径、共享关系等信息留在　　中。同一个 MemoryArea 在运行过程中可以经历未映射、只读共享和私有可写等不同页表状态，而 MemoryArea 本身保持不变。

4.3.2

用户指令给出虚拟地址后，　　先查询　　。命中时直接得到物理地址，内核不参与。未命中时，硬件页表遍历器从　　指向的根页表出发，按虚拟地址各级索引逐级查找，取得叶子项后将物理页号写入　　，再重新完成访问。

有效页表项提供物理页号和访问权限，原虚拟地址的页内偏移保持不变，最终物理地址由两者拼接而成。若映射存在且权限满足，读写或取指直接在物理内存上完成。

若页表项不存在，或者发生写入只读页面、用户态访问内核页面等权限违规时，　　产生缺页异常，将控制权交给内核的缺页处理器。此时缺页并不等于非法访问　　它也可能只是页面尚未装入，由　　按需填充后即可重新访问。

4.4

进程访问虚拟地址时，权限检查发生在两个层次：硬件在翻译阶段检查页表项标志，软件在缺页时检查 MemoryArea 权限。

4.4.1

MappingFlags 定义了四个核心权限位：READ、WRITE、EXECUTE 和 USER。MemoryArea.flags 记录这段虚拟地址应有的访问权限，是软件维护的意图描述；页表项中的对应位则是硬件实际执行检查的依据。

两层的作用不同：

MemoryArea.flags 是区域的规格说明：缺页处理时决定能否向　　请求对应权限的页面，区域权限不允许的访问直接触发 SIGSEGV，不进入填页流程。

页表项权限在硬件翻译时强制执行。没有 USER 位的页面，用户态代码无法访问，即使虚拟地址落在合法范围内；内核页面因此对用户态始终不可见，不依赖任何软件地址范围检查。EXECUTE 位（　　的　位）防止数据页被当作代码执行。

页面在父子进程共享阶段会临时去掉 WRITE 位，硬件写访问因此触发缺页，软件再按引用计数决定是复制还是直接恢复权限。权限位的这种临时降级是　　机制的触发信号，不代表区域本身变成只读。

4.4.2 mprotect

mprotect 在运行时修改已有映射区域的访问权限。内核先检查新权限是否合法　　例如，写封印的共享文件映射不允许添加 WRITE　　再调用 AddrSpace::protect。

内部实现复用与 munmap 相同的区域切分逻辑，把目标范围从已有区域中划出独立子区域，修改该子区域的 flags 并同步更新已装入页的页表项权限。MemoryArea 和页表始终保持一致：区域权限改变后，后续缺页以新权限为准，已在页表中的页面也立刻反映新标志。

PROT_NONE（全部权限清零）的常见用途是把一段地址划为守卫区。任何访问都会因页表项没有有效权限而在硬件层被拦截，产生 SIGSEGV，无需经过 MemoryArea 的软件检查。

4.5

装载程序时，信号跳板、　　各段、用户栈和用户堆共同构成进程的初始虚拟地址空间。图　　给出了　　与　　下代码中明确规定的地址位置；其中　　各段的实际范围由程序头决定，图中只标出加载基址。信号跳板映射到固定用户地址，设置为只读可执行，使用内核已有物理页，删除地址空间时只移除映射、不释放物理页。

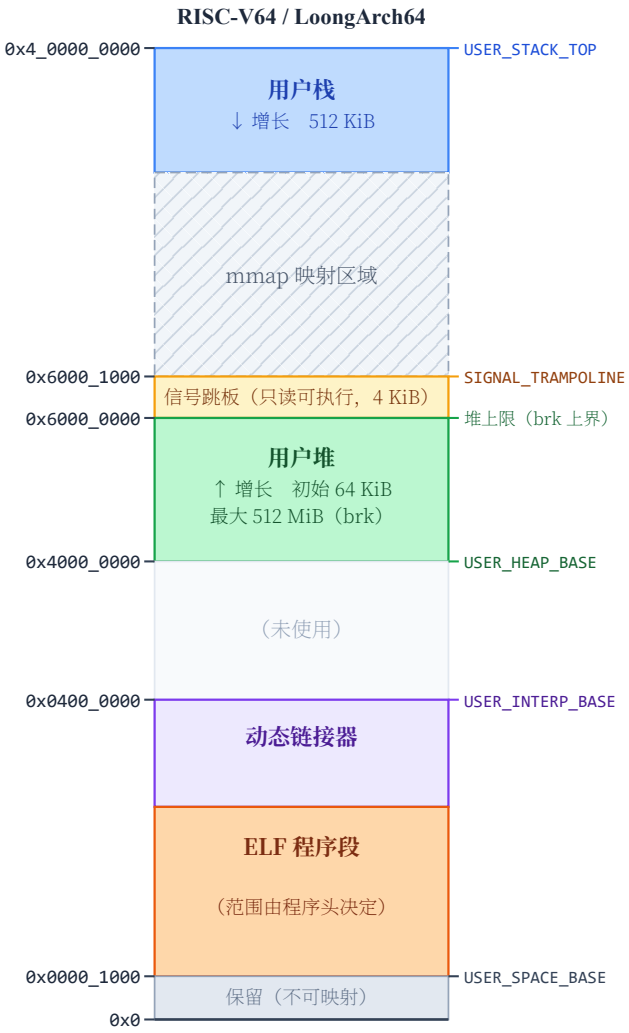


图　　用户虚拟地址空间布局

4.5.1 ELF

文件以 程序段 的形式记录装载说明，类型为 PT_LOAD 的段必须放入用户地址空间。加载器逐个处理可装载段，按　　中记录的权限建立映射：

```
for ph in program_headers.iter().filter(|ph| ph.get_type() == Ok(Type::Load)) {
    let vaddr = ph.virtual_addr as usize + elf_base;
    let seg_start = VirtAddr::from_usize(vaddr);
    let seg_pad = vaddr.align_offset_4k();
    let seg_align_size = align_up_4k(ph.mem_size as usize + seg_pad);

    let backend = Backend::new_cow(
        seg_start,
        PageSize::Size4K,
        FileBackend::Cached(cache.clone()), // ELF 文件的缓存对象
        ph.offset,
        Some(ph.offset + ph.file_size),     // 文件有效数据的结束位置
    );
```

```
    uspace.map(
        seg_start.align_down_4k(),
        seg_align_size,
        mapping_flags(ph.flags),
        false, // 只登记区域，不立即装入物理页
        backend,
    )?;
}
```

代码　　段的懒加载映射

建立映射时，内核只记录这段地址可以怎样访问，不立即把整个程序读入物理内存。代码第一次执行或数据第一次读取时，　　才分配清零页并从　　文件填入内容。文件中没有保存实际数据的　　部分因此自然保持为零。

　　在此路径中加入　　头部缓存，减少频繁 fork/exec 时重复读取程序头的开销；同时记录文件有效结束位置，避免把文件尾之后的数据当作正常内容。动态链接器、　　和　　路径兼容保证了比赛镜像中的　　、　　程序和脚本能够进入同一装载流程。

4.5.2

用户栈保存函数调用产生的局部状态。程序启动时，加载器还会将命令行参数、环境变量和辅助向量写入栈顶。内核先登记整段可读写栈区，只为启动数据实际占用的页面分配物理内存，其余页面留待首次访问时填充。

用户堆承载运行期间的动态内存分配。　　将堆起点固定为 USER_HEAP_BASE，装载程序时建立　　的初始堆，之后由 brk 调整堆顶。堆扩张时，内核为新增范围建立可读写的匿名映射；堆收缩时，解除新堆顶以外的扩展映射。栈和堆均使用　　，页面首次访问时清零分配，fork 后通过写时复制暂时共享。

4.5.3

mmap 在现有代码段、堆和栈之外建立新的虚拟内存区域。内核收到请求后，先检查映射长度、文件描述符、文件权限和映射类型，对未对齐的起始地址向下取整；固定映射（MAP_FIXED）使用调用者给出的地址，普通映射则通过 find_free_area 在地址空间中寻找连续空洞。

映射请求		行为
MAP_PRIVATE | MAP_ANONYMOUS	Cow	建立私有匿名区域，页面首次访问时清零分配
MAP_PRIVATE 文件映射	Cow	从文件读取初始内容，写入后只修改私有副本
MAP_SHARED | MAP_ANONYMOUS	Shared	预留槽位，物理页在首次访问时按需分配
MAP_SHARED 文件映射	File	多个映射通过文件缓存观察同一份内容
设备物理范围	Linear	直接映射设备提供的物理地址

表　　标志与　　的对应关系

　　补充了 MAP_SHARED_VALIDATE、MAP_FIXED_NOREPLACE、MAP_GROWSDOWN 等标志的处理，并加强文件读写权限与　　校验。MAP_LOCKED 标志被记录并统计锁定字节数，mlock 接口完成权

限和　　检查；当前不预填页或固定物理页。　　还记录每次 mmap 创建的地址范围及共享或私有属性，供 /proc/[pid]/maps 输出相应的 s（共享映射）或 p（私有映射）标记。

mremap 调整已有映射的大小：缩小时解除末尾多余的映射；扩张时优先在原区域后方追加匿名页面填充；后方空间不足且调用者设置了 MREMAP_MAYMOVE 时，内核将映射迁移到新地址　迁移路径通过 MAP_PRIVATE | MAP_ANONYMOUS 重建目标区域并复制原始内容，原有　　类型不随迁移保留。

4.6

4.6.1

程序镜像中大多数代码页在单次运行中不会被触碰，用户栈在实际增长前也不需要物理页。提前分配整段区域既浪费物理内存，也会拖慢 fork 后立即 exec 的路径　　复制了大量页面，却在　　时立刻丢弃。

AddrSpace::map 的主要工作是创建 MemoryArea。只有 populate 参数为真时才立即调用 populate_area。通常的　　段、用户栈和普通 mmap 都采用懒加载；MAP_POPULATE、栈参数和初始堆会主动填页。

populate_area 按 MemoryArea 边界遍历目标范围，对每一段调用对应　　。　　最终通过全局页分配器取得物理页，并使用页表接口建立映射。匿名页和　　副本被统计为 VirtMem 用途，释放时归还全局页分配器。

4.6.2

　　同时承担匿名分配和私有文件映射。匿名页面从全局分配器取得后清零；私有文件页面先清零，再读取该虚拟页对应的文件区间。写入私有文件页后，修改只影响进程自己的副本，不回写原始文件。

　　为共享匿名映射预留与大小对应的 Option<PhysAddr> 槽位，物理页在首次 populate 时通过 get_or_alloc 按需分配。多个进程映射到同一 SharedPages 对象时，访问同一槽位得到同一物理页。

　　根据固定偏移计算物理地址，不进行内存分配，用于将设备提供的物理范围直接接入用户地址空间。

4.6.3

　　从文件系统的页缓存取得物理页，将其映射进用户页表。页缓存以 全局索引　　组织，具体架构和写回策略已在文件系统章节说明；本节只关注内存管理侧的映射建立与一致性保证。

共享文件映射首次缺页时，　　从缓存取得对应页并建立页表映射，同时向缓存注册淘汰监听器：

```
// FileBackend 向页缓存注册回调，在缓存页被驱逐时删除用户页表项
let handle = cache.add_evict_listener({
    let this = Arc::downgrade(self);
    move |pn, _page| {
        let Some(this) = this.upgrade() else { return };
        this.on_evict(pn, &mut aspace);
    }
});
```

代码　　注册缓存淘汰监听器

缓存页被驱逐时，监听器先删除用户页表项，再完成写回和物理页释放。进程再次访问同一虚拟地址时重新缺页，　　从缓存取得新页。这一机制保证用户页表始终不持有已回收的物理页引用。

对于连续的顺序缺页，　　还会检测访问模式并触发预读，提前将后续页面装入缓存，具体策略在文件系统章节说明。

4.6.4

图　　展示了用户访问虚拟地址的完整处理路径。映射存在且权限满足时，　　直接完成访问；否则进入缺页处理，先查 `MemoryArea` 确认地址和访问类型是否合法，再调用该区域绑定的　　准备页面。

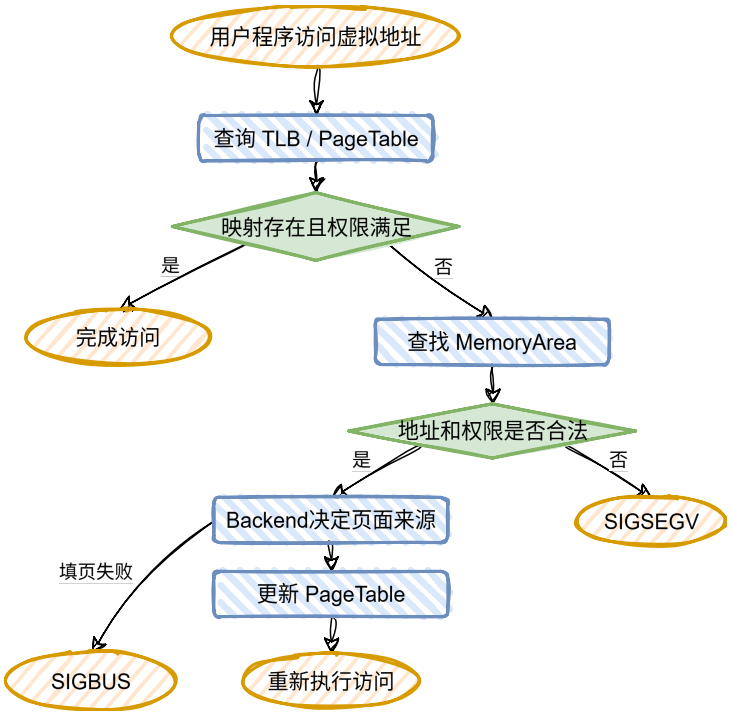


图　　用户访问虚拟地址与缺页处理流程

缺页处理集中在 `AddrSpace::handle_page_fault`：

```
if let Some(area) = self.areas.find(vaddr) {
    let flags = area.flags();
    // 区域权限必须允许本次访问类型
    if flags.contains(access_flags) {
        let page_size = area.backend().page_size();
        return area.backend()
            .populate(
                VirtAddrRange::from_start_size(
                    vaddr.align_down(page_size),
                    page_size as usize,
                ),
                flags,
                access_flags,
                &mut self.pt.modify(),
            )
            .is_ok();
```

```
    }
}
false // 地址不存在或权限不匹配
```

代码　　缺页处理的主体判断

填页成功后，用户上下文返回原指令，　　重新执行。若另一个线程已提前建立了相同映射，　　会检查现有页表权限并将该页视为已填充。

4.6.5

用户任务循环根据缺页原因选择信号：

情况	处理结果	含义
页面尚未分配，区域和权限均有效	重新填页	正常的按需分页
地址落在　　预留范围	扩展映射	为向下增长区域增加一页
地址没有对应区域或权限不符	SIGSEGV	进程访问了无效虚拟地址
合法文件映射无法提供页面	SIGBUS	地址有效，但后端存储对象不能完成访问

表　　用户缺页的处理结果

补全了 MAP_GROWSDOWN 的缺页扩展记录，并为部分文件映射和 mremap 场景维护 SIGBUS 区域。　　用户态非对齐异常还会先尝试软件模拟，无法完成时再转为 SIGBUS。

4.7

4.7.1　fork

普通 fork 调用 AddrSpace::try_clone 创建新地址空间。内核先复制父进程的虚拟区域描述，使子进程拥有相同的代码、数据、堆、栈和映射布局；随后处理已装入页表的页面。对于私有可写页面，父子进程暂时共享同一物理页，页表权限降为只读。fork 不需要立即复制整个进程内存，真正的数据复制推迟到后续写入时完成。

文件映射和共享映射在这一阶段保留原有的共享关系：文件映射继续使用同一文件缓存来源，并为子地址空间注册独立的淘汰监听器；共享内存继续指向同一组物理页。子进程因此得到与父进程一致的虚拟布局，但是否共享物理页由映射类型和后续访问决定。

带有 CLONE_VM 的　　不进入上述过程，而是让新任务直接持有原 AddrSpace，这也是同一进程内线程共享内存的实现基础。

4.7.2　COW

父子进程共享　　页面后，任意一方写入都因页表缺少 WRITE 权限而触发缺页。

查询物理页引用计数：

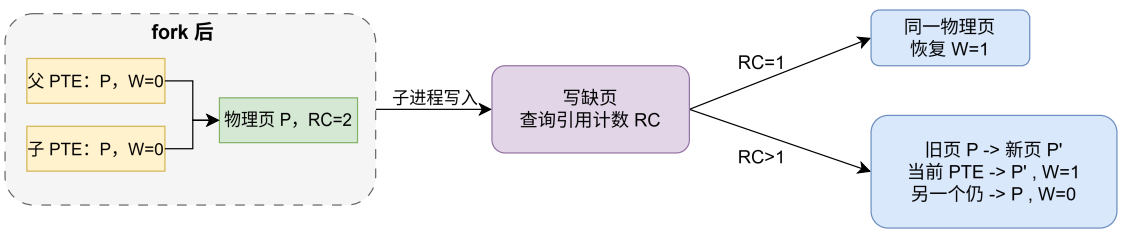


图　　的共享与写时复制

```
match frame_ref_count {
    1 => {
        // 已无其他地址空间共享，直接恢复写权限
        pt.protect(vaddr, flags)?;
    }
    _ => {
        // 仍被共享，复制数据并改映射到新页面
        let new_frame = alloc_new_frame(false)?;
        copy_page(paddr, new_frame);
        pt.remap(vaddr, new_frame, flags)?;
        drop_old_reference(paddr);
    }
}
```

代码　　写缺页的两种处理

引用数为一说明其他共享者已退出或完成复制，当前进程可直接恢复写权限。引用数大于一时，内核分配新页并复制原数据，只修改当前进程的页表。父子进程从此拥有内容相同但相互独立的页面。

4.7.3 COW

原有　　引用计数范围较小，大量 `fork` 可能造成计数溢出。　　将计数扩展为 `usize`，并在增加引用前检查溢出。

还为　　记录已装入的页面数量。一个懒加载区域如果从未发生缺页，父页表中就没有需要共享的物理页，`fork` 可以直接复制区域描述，跳过逐页查询。对于已经装入的页面，才执行只读保护、引用计数增加和子页表映射。这项修改减少了频繁 `fork/exec` 场景中的无效页表遍历。

4.8

区域删除的主路径由 `munmap` 触发。`MemorySet` 内部用有序结构保存互不重叠的 `MemoryArea`，删除一段范围时不需要逐页扫描整个地址空间。它先删除完全落在目标范围内的区域，再处理左右边界上的相交区域：覆盖区域尾部时收缩右端，覆盖头部时收缩左端，位于中间时把原区域切成左右两个。每次区域收缩或删除时，底层页表映射都会同步解除。

```
pub fn unmap(
    &mut self,
    start: B::Addr,
    size: usize,
    page_table: &mut B::PageTable,
) -> MappingResult {
    let range = AddrRange::try_from_start_size(start, size)
        .ok_or(MappingError::InvalidParam)?;

    // 删除完全位于目标范围内的 MemoryArea
    self.areas.retain(|_, area| {
        if area.va_range().contained_in(range) {
            area.unmap_area(page_table).unwrap();
            false
```

```
        } else {
            true
        }
    });

    // 左右边界的相交区域：收缩或切分，各自同步清除页表项
    ...
}
```

代码 的区域删除逻辑

mprotect 与 mremap 复用了这套区域切分能力：mprotect 不删除区域，只在切分后修改目标子区域的权限和页表标志；mremap 缩小时调用 unmap 删除末尾范围，扩张时优先在原区域后方追加匿名填充，空间不足且允许移动时迁移到新地址。brk 收缩也会进入区域删除路径，但只解除初始堆区之外的扩展页面。execve 和地址空间析构调用 AddrSpace::clear，一次性删除全部区域。

AddrSpace 实现了 Drop，进程最后一个地址空间引用消失时自动调用 clear。因此，正常退出、创建失败后的对象回收和 替换最终都汇入同一套区域删除与 释放流程。

4.9

的内存管理以 AddrSpace、MemoryArea、PageTable 和 四个结构为核心。地址空间先登记可用区域，页面由 按需装入；多级页表负责虚拟地址翻译，USER 和 EXECUTE 等权限位在硬件层隔离用户态访问；缺页处理连接区域描述和物理页面；mprotect 在运行时修改区域权限并同步页表；fork 根据 类型选择共享、 或线性重建，私有可写页面通过引用计数驱动的 延后复制。

在 基础上补全了若干细节：mmap 对未对齐起始地址向下取整；MAP_LOCKED 接受请求并统计锁定字节数；mremap 迁移路径通过匿名私有映射重建，原 不随迁移保留；msync 目前只作为接口桩存在，脏页写回依赖 fsync 和全局 sync 触发； 引用计数扩展为 usize 并加入溢出检查。文件页在用户页表中的接入与淘汰同步由 负责，缓存架构和写回策略已在文件系统章节说明。

5

5.1

正如众计算机学子所周知的　　设计哲学　　一切皆文件　。文件系统要解决的核心问题是：让用户程序用同一套接口（　　）去访问性质完全不同的对象　　磁盘上的文件、内存里的临时文件、内核生成的　　条目和　　下的设备节点。这些对象的存储方式、内容来源和访问语义差异很大，但用户程序不该感知这种差异。

　　的文件系统沿用　　的做法分四层来处理这个问题。最底层　　只定义接口：文件系统是什么、目录节点能做什么、文件节点能做什么，不涉及任何存储细节。　　在此之上实现路径解析、页缓存和具体的　　后端。再往上，　　提供　　兼容的 `FileLike` 接口，把　　的文件对象、管道、　　、设备文件统一进一张文件描述符表。最顶层　　负责把系统调用的参数翻译成对上层接口的调用。

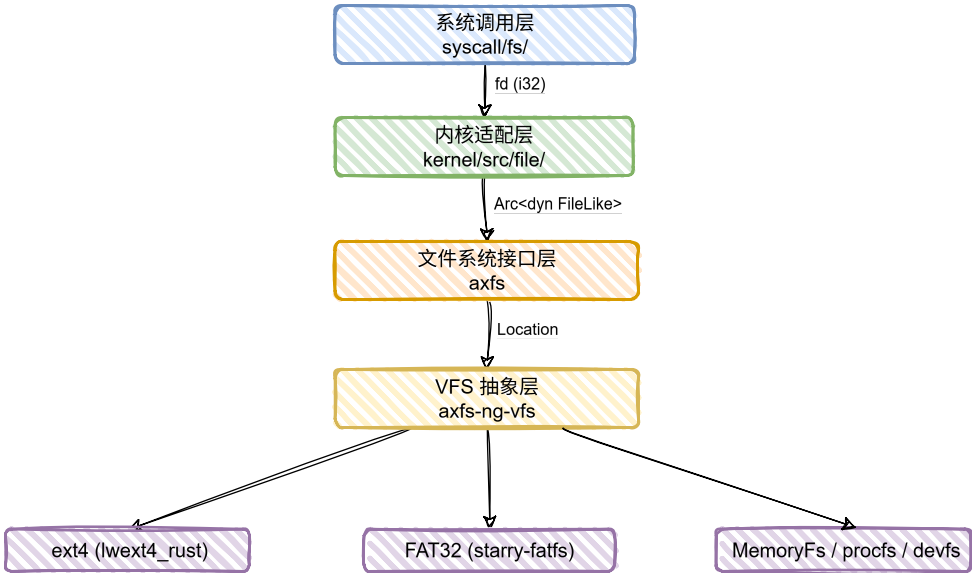


图　　文件系统分层架构

这种分层的好处是职责清晰：路径解析、挂载语义、权限检查在各自的层处理，不会混入具体文件系统的实现里。　　提供了这套框架的雏形，　　的工作主要集中在挂载命名空间的完整化、页缓存的统一管理、　　多路复用和文件事件通知上。

5.2

没有挂载，一切皆文件 就只是句口号。磁盘分区、插入的　盘、内核生成的　　、内存里的临时目录　　它们生来是独立的存储对象，各自有自己的根，根本不认识彼此。挂载解决的就是这个问题：把独立的文件系统动态拼到同一棵目录树下，让 `/home/user` 和 `/proc/meminfo` 可以用同一套接口访问，调用方完全不需要知道底下的存储是什么。

在内核里，每次 `mount` 调用都在一棵 `Mountpoint` 树上添加一个节点，记录 这个文件系统的根接在了哪个目录项上 。路径解析穿越目录时，一旦碰到挂载点就自动切换到对应文件系统的根　　这个切换对调用方透明，`open("/mnt/foo")` 和 `open("/bin/bash")` 走同一套代码路径，差别只在于前者在路途中换了一次 文件系统 。

5.2.1

`sys_mount` 除了普通挂载外，还支持几种改变已有挂载关系的操作模式：

标志	含义
MS_BIND	：让已有目录树在另一条路径下可见，不复制任何文件
MS_MOVE	迁移挂载点：将挂载从一个路径移到另一个路径，不卸载重挂
MS_REMOUNT	修改现有挂载的标志，如将读写挂载改为只读
MS_PRIVATE	命名空间隔离：本挂载点的变动不向父 子命名空间传播

表 支持的操作模式

值得单独说一下。它不创建新的文件系统实例，只是让同一棵目录树从另一条路径也可以访问到。容器场景里常用这个：把宿主机的某个目录以只读方式 进容器，容器内用户能读到内容，但任何写操作都会被挂载标志拦下来，底层数据不受影响。 的实现直接复用源挂载点对象，新的 Location 指向同一个 Mountpoint， 天然相同，stat() 结果与 一致。

sys_umount2 卸载时会向所有关注该挂载点的 发送 事件，监视的应用能在文件系统被卸载时收到通知。

5.2.2 API

传统 sys_mount 有个历史包袱：调用者必须同时拥有源设备的访问权限和目标路径的挂载权限，两者缺一不可。在容器场景里这很别扭 容器进程应该能在自己的命名空间里自由挂载，却因为没有宿主机路径的权限而卡住。

针对这个问题引入了基于 的新式挂载 。 没有实现这套接口，参考 内核的语义补上了这部分，把挂载拆成几个独立步骤：

```
fd  = fsopen("ext4")                        // 创建文件系统上下文
      fsconfig(fd, "source", "/dev/loop0")  // 配置设备路径
      fsconfig(fd, "cmd", "create")          // 完成配置，准备挂载
mfd = fsmount(fd, 0, 0)                     // 实例化为挂载对象 fd
      move_mount(mfd, "/mnt")               // 将挂载接入目录树
```

代码 新式挂载 的调用流程

关键在于 fsopen 到 fsmount 这几步可以在无特权的用户命名空间里完成，只有最后的 move_mount 才需要对目标路径的权限。open_tree 反向工作，把现有的挂载子树克隆成一个 ，可以跨命名空间传递再接入。这五个调用共同构成了容器运行时和命名空间工具所依赖的挂载语义基础。

5.3 VFS

5.3.1

路径解析看起来简单 逐级查目录 但挂载的存在让它变得复杂。/mnt 可能挂载了一个 镜像，/mnt 里的某个子目录又做了 。穿越挂载点时， 当前位置 不只是一个目录项，还包括它处于哪个挂载上下文里 因为挂载标志（只读、 等）是挂载级别的属性，同一个目录项在不同挂载下行为可以不同。

用 Location 把这两者绑在一起：

```
pub struct Location {
    mountpoint: Arc<Mountpoint>,  // 当前所属挂载点
    entry: DirEntry,              // 目录树节点
```

```
}

pub struct Mountpoint {
    parent: Option<Location>,      // 挂附在哪里
    children: Mutex<HashMap<ReferenceKey, Weak<Mountpoint>>>,
    device_id: DeviceId,
    flags: MountFlags,
}
```

代码 ：将目录节点与挂载上下文绑定为统一句柄

`Location` 作为路径解析过程中所有操作的句柄，有几个直接的好处： 不需要额外的数据结构，直接复用源挂载点对象，`stat()` 返回的 便自然正确；堆叠挂载（同一目录连续 多次）通过 `effective_mountpoint()` 向上遍历到最顶层，这样调用方便看不到底层细节； 的 `link` 和 `rename` 用 `Arc::ptr_eq` 对比两个 `Location` 的挂载点指针即可，不需要额外的设备号比较逻辑。

`Mountpoint` 树的子节点用 `Weak` 引用而非 `Arc`，这个选择是为了让卸载操作自然生效 子挂载点没有外部引用时会自动释放，不需要显式通知父节点，也可以避免父子引用形成环，影响挂载点的释放。卸载前的 检查会反过来用 `Weak::strong_count` 判断子挂载点是否还活着，如果这个挂载点还在被使用的话则拒绝卸载返回错误码 。

图 展示了一个典型的挂载点树。 挂载在根， 、 、 作为三个并列的子挂载点分别挂载 、 和 镜像，父到子的引用均为 。 下还有一个 到 ，以只读标志重新暴露同一份 数据，这个子挂载点用 引用持有，卸载 时它会随之自动释放。

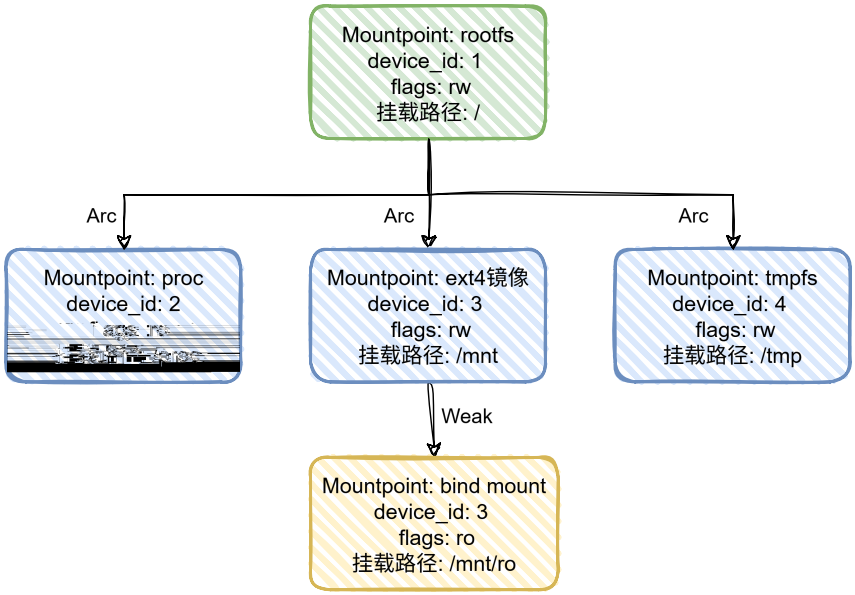


图 树

5.3.2

把 、 节点、普通文件、设备节点统一抽象为 文件节点 ，上层就需要知道怎么对待它们的差异。一种做法是在 里做类型判断，但这样每增加一种节点类型都要修改上层，很不利于代码的实现。 的做法是让节点自己声明 契约，用 `NodeFlags` 表达。

标志	含义	使用方
STREAM	没有文件偏移的概念，读写始终从头开始	、
NON_CACHEABLE	内容动态生成，不能缓存元数据	、
ALWAYS_CACHE	强制走页缓存，没有块设备后端	
BLOCKING	直接阻塞，不包　　等待循环	普通文件

表　　：节点向上层声明自身　　契约

有了这些标志，　　就能做出正确的决策：STREAM 节点不维护文件偏移；NON_CACHEABLE 节点每次　　都走后端；ALWAYS_CACHE 节点的写入不需要回写到任何设备。上层不再需要　　判断节点类型，节点的差异被限制在声明这一步。

元数据缓存以 (fs_ptr, inode) 为键，存入了全局的 BTreeMap，这样多个进程、多个　、多个路径访问同一个文件时，可以共享同一份缓存，提高访问效率。由于目录的　　会随子节点增删而变化，缓存它很容易导致读到过期数据，所以将目录节点被刻意排除在元数据缓存外。

5.3.3　　FsContext

路径解析需要处理几个彼此独立的问题：每个进程有自己的工作目录和　　根；符号链接可能形成循环；穿越挂载点要切换文件系统上下文。

用 scope_local! 给每个任务绑定一个独立的 FsContext，里面存着 root_dir 和 current_dir。scope_local! 是　　提供的机制，类似线程局部存储，但生命周期和任务绑定，　　时从父任务继承。root_dir 既是　　的锚点，也是路径向上遍历的上界　　遇到 .. 到达根之后继续向上，会被静默截断，这正是　　应有的行为。

符号链接的循环检测靠一个共享计数器 follow_count，它以可变引用的形式贯穿整个解析调用栈。　　这样的间接链，三次跳转都计入同一个计数器，不会因为 每次只跟一跳 而绕过上限。上限设为　，与　　一致，超过就返回错误，避免死循环。

解析本身有四个变体，对应不同的　　语义：

变体	适用场景
resolve	跟随末端符号链接，　　等常规路径
resolve_no_follow	不跟随末端符号链接，
resolve_nonexistent	父目录必须存在，末端必须不存在，
resolve_parent	返回父目录和末端名称二元组，

表　　路径解析的四个变体覆盖所有　　场景

每次目录查找完成后，　　自动调用 resolve_mountpoint() 检查是否到达挂载点，如果是则切换到对应文件系统的根。这个操作对调用方透明，穿越挂载点和普通的目录查找在接口上没有区别，可以让所有系统调用都共享同一套正确路径语义。图　　展示了一次完整路径解析的处理流程。

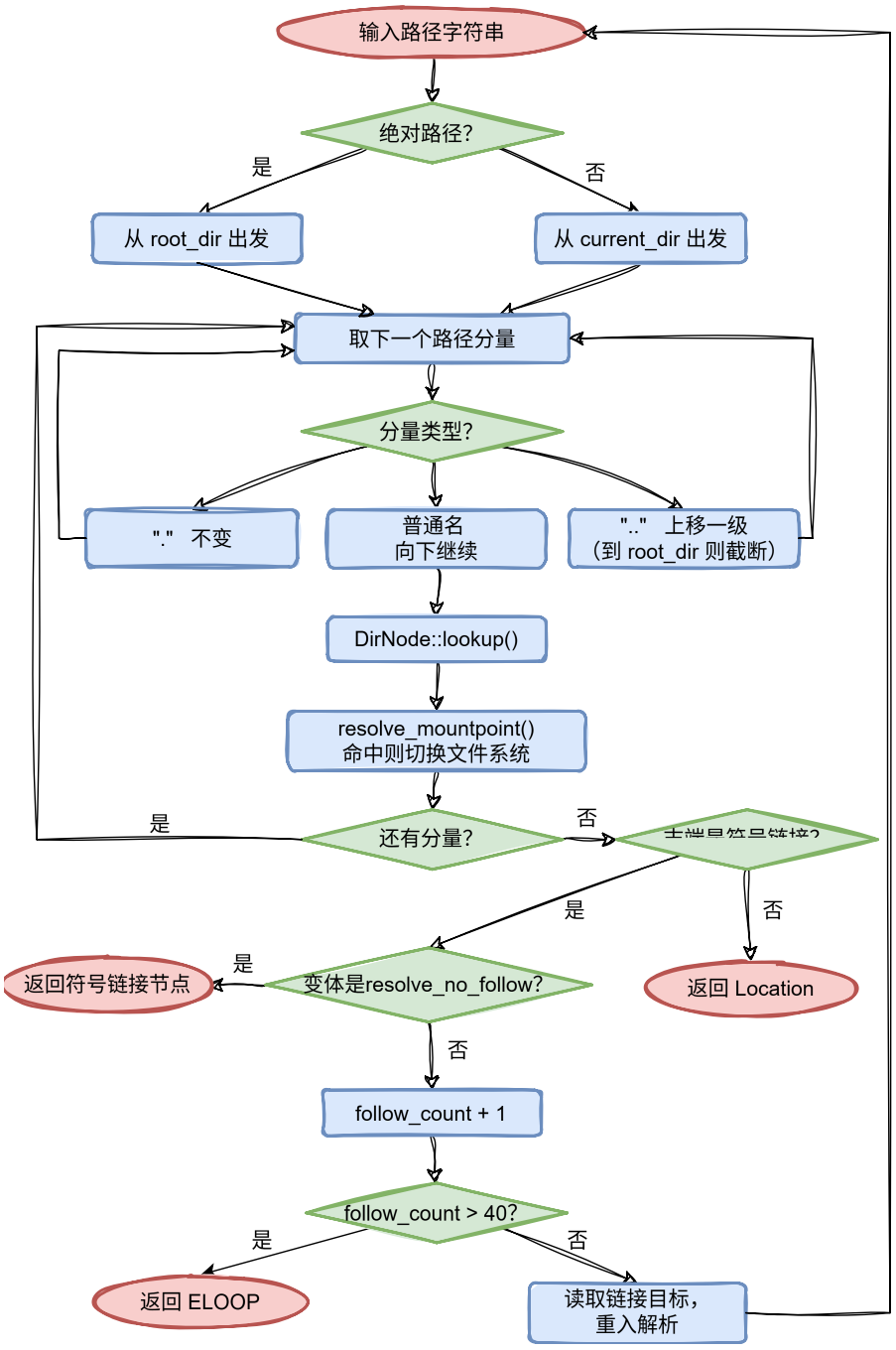


图　　路径解析的完整处理流程

5.4

5.4.1

是承载根文件系统和用户程序的主力，　　通过 `lwext4_rust`（　　库的绑定）接入，底层块访问通过 `BlockDriverOps`　　向下适配。这里有一个实际的测试问题：中很多用例会在运行时用 `mount` 把一个镜像文件挂载成　　，然后在里面做文件操作测试。要支持这个场景，需要能把普通文件当成块设备用。

为此实现了 `LoopFileBlockDevice`：

```
pub struct LoopFileBlockDevice {
    // 被当作块设备使用的普通镜像文件。
    file: Arc<Mutex<File>>,
```

```
    // ext4 后端看到的逻辑块大小。
    block_size: u32,
}

impl BlockDriverOps for LoopFileBlockDevice {
    fn read_block(&mut self, block_id: u64, buf: &mut [u8]) -> ... {
        let offset = block_id * self.block_size as u64;
        self.file.lock().read_at(offset, buf)
    }

    fn write_block(&mut self, block_id: u64, buf: &[u8]) -> ... {
        let offset = block_id * self.block_size as u64;
        self.file.lock().write_at(offset, buf)
    }
}
```

代码 将普通文件适配为块设备接口

这层适配的核心就是块号到文件偏移的换算：块设备第 块，对应普通文件中从 `N * block_size` 开始的一段内容。 后端仍然按块读写，`LoopFileBlockDevice` 则把这些块访问翻译成对镜像文件固定偏移处的 `read_at` `write_at`，这样普通文件就具备了块设备接口。

`sys_mount` 识别 `/dev/loop*` 路径时，把对应文件包装成 `LoopFileBlockDevice`，再传给后端。这样镜像文件挂载就走普通的 挂载路径，不需要单独处理。

基于 `starry-fatfs`（ 基础层的纯 实现）接入，底层负责解析 表和目录项，上层仍然通过同一套 接口执行 等操作。 本身只保存文件名、大小、目录关系和只读、隐藏等简单属性，没有 的 、 权限位和完整 模型，因此适配层在返回元数据时为这些字段补出默认值：普通文件和目录给出固定的 ，使用挂载时约定的默认身份， 等字段也按 需要合成。这样上层程序即使访问 ，也能像访问 一样拿到一份 风格的 结果。

5.4.2

的核心需求是：快速创建销毁文件，不访问磁盘，支持 和 这类高频场景。 实现的 `MemoryFs` 在 管理和文件内容存储上都做了针对性的设计。

用 `slab::Slab<Arc<Inode>>` 分配。 分配的好处是 号直接等于 下标加一，这样分配和按号查找都是 ，不需要维护额外的 映射表。

核心代码如下：

```
pub struct MemoryFs {
    // Slab 像一张可复用空槽的 inode 表。
    inodes: Mutex<Slab<Arc<Inode>>>,
    // ...
}

impl MemoryFs {
    fn get(&self, ino: u64) -> Arc<Inode> {
        // inode 号从 1 开始，Slab 下标从 0 开始。
        self.inodes.lock()[ino as usize - 1].clone()
```

```
    }
}
```

代码　　用　　分配　　并合成　　号

因此，创建文件时只需要从　　取一个空槽，空槽下标加一就是对外暴露的　　号；之后按　　号查找时再减一，就能直接定位回　　中的对象。

文件内容则反向思考：MemoryFs 的　　里不存数据，FileContent 只记录一个 length: Mutex<u64>，实际内容全部委托给上层页缓存（靠 ALWAYS_CACHE 标志触发）。这样做有两个好处：

本身非常轻量，不需要在内存中维护两份缓冲区；全零写入由页缓存检测，不分配缓存页，天然支持稀疏文件。

释放　　时要同时满足 nlink == 0 和 Arc::strong_count == 2（一份在　　，一份在当前调用者）才真正释放。只满足前者不够　　进程可能还持有打开的文件描述符，此时　　虽然没有目录项指向它，但数据不能提前销毁。

5.4.3

　　和　　解决的是一个本质问题：　　里的文件内容由内核按当前状态动态生成，下的设备文件背后是驱动而不是存储，但两者都需要出现在文件系统的路径空间里，能被普通的　　访问。

　　用 SimpleFs 框架处理这类伪文件系统，它提供　　分配和基本的目录结构，具体的目录内容和文件读取逻辑通过 SimpleDirOps　　注入。　　和　　只需要实现 这个目录下有哪些条目 和 读这个文件返回什么 ，其他的框架处理。两者的目录结构如下：

```
/proc                                  /dev
├── [pid]/                             ├── null    ← SPECIAL_NODES 注入 rdev
│   ├── stat                           ├── zero    ← SPECIAL_NODES 注入 rdev
│   ├── status                         ├── urandom
│   ├── maps                           ├── tty
│   ├── pagemap                        ├── ptmx    ← PTY 主设备
│   ├── cmdline                        ├── pts/    ← PTY 从设备目录
│   ├── exe  ──► 可执行文件路径          │   ├── 0
│   ├── fd/                            │   └── 1
│   └── ns/  (pid/uts/user)            ├── loop0   ← loop 块设备
├── self/ ──► 当前进程 [pid]/           ├── fb0     ← 帧缓冲
└── sys/                               └── rtc0    ← 实时时钟
    ├── kernel/
    ├── fs/   (pipe-max-size 等)
    └── vm/   (drop_caches 等)
```

代码　　与　　的目录结构

　　中 [pid]/ 目录是动态的，每个运行中的进程对应一个。其中有个容易出错的地方：进程可能在　　条目被访问的同时退出。ProcessTaskDir 用 Weak<Process> 持有进程引用，升级 Weak 失败时直接返回空目录，不会悬空也不会　　。

　　里 /dev/null、/dev/zero 等设备文件在真实文件系统上并不存在，但　　时要返回正确的　　号。　　用全局 SPECIAL_NODES 表处理这个问题　　调用时检查路径是否在表里，如果是则用表项中的　　和　　覆盖底层节点的返回值。这个覆盖发生在　　层，不需要修改任何文件系统后端。

5.5 I/O

文件读写请求经过 CachedFile 后端统一处理。每个文件在全局缓存表中有一份 CachedFileRecord，以 (fs_ptr, inode) 为键索引，同一文件被多个进程或多个 打开时共享同一份缓存记录，不会出现多份副本。读写请求根据文件类型走三条路径：普通磁盘文件走 路径，经页缓存层访问；O_DIRECT 打开的文件走 路径，绕过缓存直接访问块设备；等内存文件系统走 路径，不涉及块设备。三条路径的分流结构如图 所示。

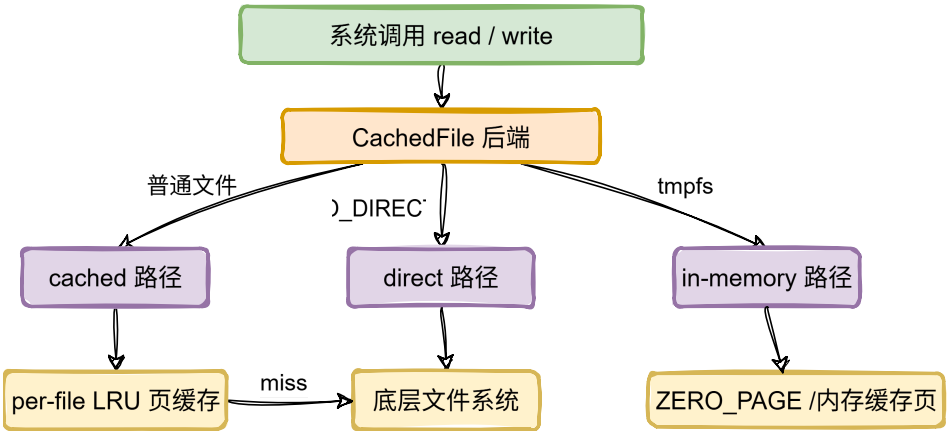


图 读写三条路径

5.5.1

采用 全局索引 的结构管理页缓存：全局表以 (fs_ptr, inode) 为键记录各文件的缓存入口，真正的数据页缓存在每个文件自己的 里，默认容量 个 页( )。这样设置有两个好处：单个文件的大规模顺序扫描不会把其他文件的缓存挤出；对于能完整装入缓存的小文件，一旦内容被读入，后续的重复读、非顺序访问都可以直接命中缓存，无需再访问底层存储。

这个结构的关键代码如代码 所示。全局表只负责 同一个文件找到同一份缓存记录 ，页数据本身仍然放在每个文件自己的 CachedFileShared 里。

```
// vendor/axfs/src/highlevel/file.rs
// ...
type FileCacheKey = (usize, u64); // (fs_ptr, inode)，唯一定位某个文件

struct CachedFileRecord {
    key: FileCacheKey,              // 当前缓存记录对应哪个文件
    location: Location,             // 文件在 VFS 中的位置，用来回到底层读写
    shared: Arc<CachedFileShared>, // 真正共享的页缓存和同步状态
    in_memory: bool,                // 是否是 tmpfs 这类内存文件
}

static FILE_CACHES:
    // 全局索引表：同一文件的多个 fd 会找到同一个 CachedFileRecord
    Lazy<RwLock<BTreeMap<FileCacheKey, Arc<CachedFileRecord>>>> = ...;
// ...
```

代码 文件页缓存的全局索引记录

走　　路径：它也通过全局索引找到自己的缓存记录，但记录中的 `in_memory` 为　，内部使用无界缓存，且不需要写回块设备。读未分配页时直接返回全局 `ZERO_PAGE` 中对应的内容，写入全零数据时不分配真实缓存页，能够支持稀疏文件或者全零写入。

写路径做了一项针对性的优化，避免　　。页缓存以　　为单位操作，朴素的实现在目标页缺失时会先读出完整旧页再部分覆盖　　这对修改已有文件的中间段是必要的，但对新文件顺序写或　　之后　　而言，旧内容根本不存在或即将全部被覆盖，先读旧页纯属浪费。　　用 `can_zero_fill_write_page` 判断：如果当前页完全在旧　　之后，或者本次写入从页内偏移　开始且覆盖了页内所有旧有效内容时，直接分配清零页而不调用底层 `read_at()`，消除了新页写入前的无效读取。

脏页写回时，所有脏页按页号排序，连续页合并成一批写入，每批最多　页（　　）。如图　所示，相比逐页提交，顺序写场景下的块设备请求数量可以明显减少，对　　、　　等需要落盘的路径尤为有效。

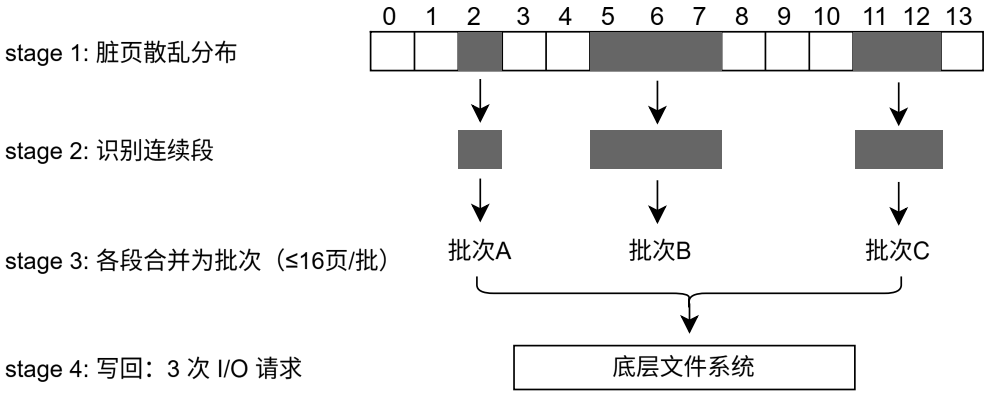


图　　脏页按页号排序后合并为批次写回

读写绕过页缓存，但同一文件可能同时有　　在访问，如果进程　用　写了数据，还在缓存里没落盘，此时进程　又要用　　读同一段，如果不处理，　可能读到旧数据。为维护一致性，　　在　　读前先将目标范围内的　　页写回，写后使对应干净页失效，这样两条路径就不会读到不一致的内容。

5.5.2

频繁的　　调用和读写带来的时间戳更新如果每次都穿透底层文件系统，会在　　密集场景产生大量元数据写。如果一个任务只是想读文件，却要附加一次写的任务，就会产生很多无用的开销。为解决这一问题，　　在　　层引入　　元数据缓存，以 `(fs_ptr, inode)` 为键将　、　、　、　缓存在内存中。目录的　　随子节点增删变化，缓存容易读到过期值，被排除在外；`NON_CACHEABLE` 节点（　、　等）内容动态生成，同样不进入缓存。

元数据缓存的核心字段如代码　所示。它只保留当前报告需要理解的字段：一份缓存后的 `metadata`，以及两个表示 还有东西没提交到底层 的　　标记。

```
// vendor/axfs-ng-vfs/src/node/mod.rs
// ...
type InodeMetadataCacheKey = (usize, u64); // (fs_ptr, inode)

struct InodeMetadataCache {
    metadata: Metadata,            // 当前内存里缓存的 size/atime/mtime/ctime 等
    dirty_size: bool,              // 文件大小是否已经改了但还没提交
    dirty_update: MetadataUpdate,  // 时间戳、权限、所有者等待提交更新
```

```
}

static INODE_METADATA_CACHES:
    // 全局元数据缓存表：同一 inode 的 stat/fstat 共享这一份结果
    Lazy<Mutex<BTreeMap<InodeMetadataCacheKey, SharedInodeMetadataCache>>>
= ...;
// ...
```

代码　　元数据缓存记录

元数据更新采用延迟提交策略：各字段的修改先合并到 dirty_update，不立刻落盘；　　或文件关闭时统一提交，把多次独立的元数据写合并为一次。　　更新遵循　　策略：只有　　早于　　或　　、或距上次更新超过　小时，读操作才会触发　　更新，大多数只读访问因此不再产生元数据写。MS_NOATIME 和 MS_STRICTATIME 挂载标志可以覆盖这个策略。

5.5.3

预读在应用程序实际访问页面之前提前将文件内容放入页缓存，降低后续访问的等待时间。　　基线的 fadvise(WILLNEED) 只做参数检查，不实际预读。　　补全了这条路径：sys_readahead() 和 fadvise(POSIX_FADV_WILLNEED) 找到文件对应的　　，按请求范围同步地将页面放入缓存，单次最多　　。　　场景保持　　，符合绕过缓存的语义。

　　访问时每次　　只处理当前缺失页。　　在　　后检查 last_fault_page：若当前页号比上次大 ，说明是顺序扫描，额外将后续　页预热入缓存。预读只把文件页放入页缓存，不建立页表映射，后续　　命中缓存时等待时间更短，且不影响随机访问场景下单次　　的延迟。

共享匿名映射的物理页采用懒分配：初始化时只创建占位槽，首次　　时才申请物理页，未被访问的页不占用真实内存。

5.5.4 I/O

服务器程序需要同时等待多个　上的事件，轮询是一种做法但　　的开销极大。正确的实现应该是：没有事件时挂起任务，有事件时精准唤醒。

　　实现了　　、　　和　　三种接口。　　和　　的实现相对直接：通过 block_on(timeout(poll_io(...))) 把当前任务交给调度器，等到有　就绪或超时时再返回，全程不占用　　。

　　需要更精细的控制，因为它要在　就绪时立刻通知，而不是等调用者再次轮询。　的实现里，每个被监视的　对应一个 EpollInterest，持有一个实现了 std::task::Wake 的 InterestWaker。当底层文件对象（　、　、设备等）的状态变化时，它调用 InterestWaker::wake()，后者把这个　　推入　　的　　，并唤醒阻塞在 epoll_wait 上的任务。整条路径是事件驱动的，没有任何轮询。

　　的核心状态可以见代码　　这一段：一张 关注表 记录用户关心哪些　，一个　记录已经就绪的　，再加一个 PollSet 用来唤醒睡在 epoll_wait 上的任务。

```
// kernel/src/file/epoll.rs
// ...
struct EpollInner {
    // 关注表：key 是被监听的 fd，value 是该 fd 关心的事件和触发模式
    interests: SpinNoPreempt<HashMap<EntryKey, Arc<EpollInterest>>>,
```

```
    // 就绪队列：底层 fd 有事件时，把对应 interest 放进这里
    ready_queue: SpinNoPreempt<VecDeque<Weak<EpollInterest>>>,
    // 等待队列：ready_queue 从空变为非空时唤醒 epoll_wait
    poll_ready: PollSet,
}
// ...
```

代码 的关注表与就绪队列

还支持三种触发模式：

模式	行为
（电平触发）	就绪时持续上报，直到数据被读尽，默认模式
（边沿触发）	仅在状态从未就绪变为就绪时通知一次，要求应用读尽数据
	触发一次后自动撤除监听，需显式重新 才能继续接收事件

表 的三种触发模式

嵌套 （把一个 加入另一个 ）需要防止形成环形等待。 在 epoll_ctl 时递归检查，深度上限 层，同时记录已访问的实例指针来发现环。

ppoll 和 pselect6 需要原子地切换信号掩码然后进入等待 如果先换掩码再等待，中间有个窗口期可能丢失信号。 在进入 block_on 前完成掩码切换，返回时恢复，保证原子语义。

5.6

5.6.1

系列（ ）返回固定字段集合。 参考 语义实现了扩展版本 ，在兼容 全部字段的基础上增加了两类能力：文件系统无法提供的字段（如 stx_btime 创建时间）可以用 STATX_ATTR_* 标志标记为 不可用 而不是返回错误；stx_mnt_id 返回挂载 ，让调用者知道这个文件属于哪个挂载点。SPECIAL_NODES 表中的设备文件在 时用表项值覆盖 和 。

接口	字段集合	实现要点
stat/fstat/lstat	固定字段集	设备文件的 由 SPECIAL_NODES 表覆盖
statx	可选扩展字段	STATX_ATTR_* 标记不可用字段；stx_mnt_id 报告挂载点

表 与 的能力对比

5.6.2

access 和 faccessat2 看起来只是查 位，实际要处理几条叠加规则，还要区分使用哪套 。 把这些都做全了：

规则　接口	返回值	说明
父目录搜索权限		路径每一级父目录都须通过　　，缺一即拒
只读挂载写请求		写权限被挂载标志拦下时返回　　，不是
执行位要求		执行文件须至少一个　位，与普通用户不同
access		反映调用者真实身份，　　程序中是原始调用者
faccessat2 AT_EACCESS		反映当前运行身份，　　程序中是文件所有者

表　　权限检查的叠加规则与　　语义

不区分　　与　　，　　补上了这一差异。

（FS_IMMUTABLE_FL、FS_APPEND_FL 等）不依赖文件系统后端，　　用全局 INODE_FLAGS 表按路径存储，FS_IOC_GETFLAGS FS_IOC_SETFLAGS　　读写，　　和写操作时查表叠加到检查逻辑里。　　在　　上有完整实现，　　通过　　接口透传。

5.7

用户程序有时需要知道文件系统上发生了什么　　备份工具要感知哪些文件被修改，安全软件要在文件被访问时介入。　　提供了两套机制，能够覆盖不同的使用场景，　　对此也进行了实现。

5.7.1 inotify

面向具体文件或目录的变化监视。用户态对目标路径添加　　，之后　　、　　、　　、　　等操作产生的事件进入事件队列，通过　　取出。移动操作会产生成对的　　和　　事件，两者通过　　字段关联，让接收方能识别是同一次　　。

实现上，一个　　维护两张表：watches 记录用户关注哪些　　，events 保存已经发生但尚未被　　取走的事件。

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
struct Target {
    dev: u64,     // 文件系统设备号
    ino: u64,     // inode 号，和 dev 一起唯一定位目标
    is_dir: bool, // 事件目标是否是目录
}

#[derive(Clone)]
struct Watch {
    wd: c_int,       // 用户态看到的 watch descriptor
    target: Target,  // 被监视的文件或目录
    mask: u32,       // 关注的事件集合，如 IN_CREATE/IN_DELETE
}

struct Event {
    wd: c_int,              // 事件对应哪个 watch
```

```
    mask: u32,              // 实际发生的事件
    cookie: u32,            // rename 时关联 MOVED_FROM/MOVED_TO
    name: Option<String>,   // 目录事件中的子项名称
}

pub struct InotifyFd {
    watches: Mutex<Vec<Watch>>,       // 当前 fd 上注册的所有 watch
    events: Mutex<VecDeque<Event>>,   // 等待用户态 read 的事件队列
    poll: PollSet,                    // 新事件到达时唤醒 read/poll
}
```

代码 : 表和事件队列

图 展示了 从注册 到用户态取出事件的完整数据流，其中 产生的 与 通过相同的 值关联为一对。 在 的创建、删除、重命名和属性更新路径上插入通知点，卸载时向相关 发送 。

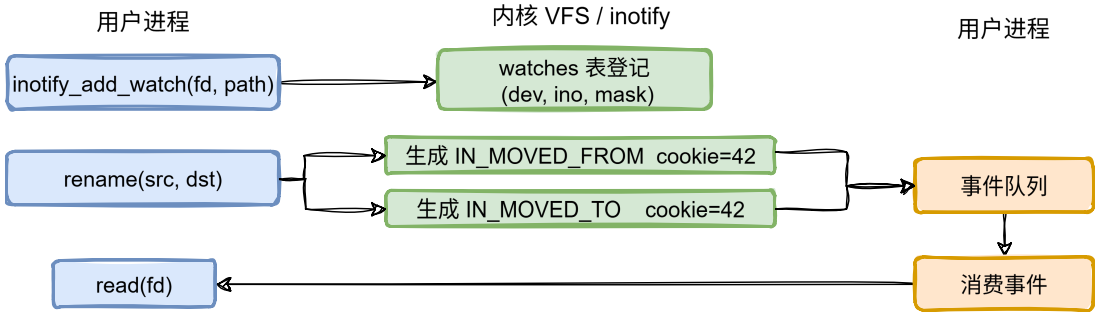


图 事件投递流程

5.7.2 fanotify

解决的是 解决不了的两个问题：监视范围更大（可以作用在单个 、整个挂载点乃至整个 ），以及能在访问发生时主动介入，而 更像是一种事后通知。

可以挂在单个文件、目录或整个挂载点上， 指定关注的事件类型。 模式下，事件报告中携带文件系统标识符和文件句柄，监听者无需持有路径就能定位文件 这对跨挂载点的全局监视很重要，因为文件可能在监听期间被移动过。

的核心也是 关注集合 事件队列 ，但 的作用范围更丰富，事件中还可能携带文件句柄、子项名称和权限请求编号。

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
enum FanotifyMarkKind {
    Inode,      // 监视单个文件或目录
    Mount,      // 监视整个挂载点
    Filesystem, // 监视同一文件系统
}

struct FanotifyMark {
    target: FanotifyTarget, // 被监视的 inode/挂载点/文件系统
    path: Option<String>,   // 需要按路径报告时保留别名
    kind: FanotifyMarkKind, // mark 的作用范围
    mask: u64,              // FAN_OPEN/FAN_ACCESS/FAN_CREATE 等
```

```
    ignored_mask: u64,      // 临时忽略的事件集合
}

struct FanotifyEvent {
    mask: u64,                      // 实际触发的 fanotify 事件
    file: Option<Arc<dyn FileLike>>, // 需要返回 fd 时保留文件对象
    fid: Option<FanotifyTarget>,    // FID 模式下报告的文件身份
    name: Option<String>,           // 目录项事件的子项名称
    permission_id: Option<u64>,     // 权限事件等待用户态响应
}

pub struct FanotifyFd {
    marks: Mutex<Vec<FanotifyMark>>,        // 当前监听实例上的所有 mark
    events: Mutex<VecDeque<FanotifyEvent>>, // 等待读取的事件队列
    permissions: Mutex<Vec<FanotifyPermission>>, // 尚未 allow/deny 的权限事件
    poll: PollSet,                          // 唤醒 read/poll 等待者
}
```

代码　　：　　、事件队列与权限等待表

权限事件（FAN_OPEN_PERM、FAN_ACCESS_PERM）是　　最特殊的部分：内核在文件被打开或访问时，先等待监听进程的响应，再决定是否让操作继续。触发进程在等待期间阻塞，监听进程通过写入　　或　　响应。安全软件（防病毒、　　）依赖这个机制在文件访问路径上做拦截。图　　展示了这一阻塞 响应流程中三方的交互时序。

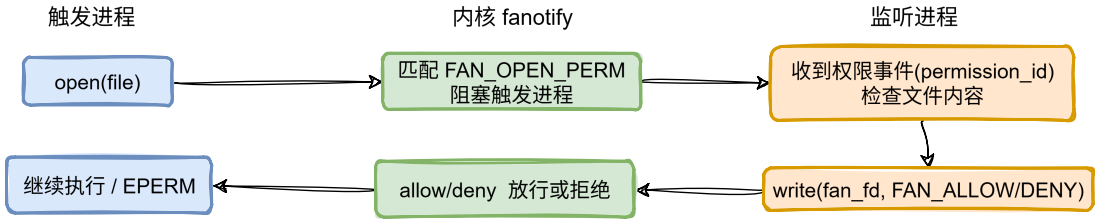


图　　权限事件时序

　　的　　和　　实现覆盖了打开、读写、创建、删除、重命名、关闭等路径上的通知点。

5.7.3

　　和　　的实现需要在　　、　　、　　、　　等系统调用路径上插入事件投递逻辑。没有注册监听者时，这些逻辑不应带来任何额外开销。

5.8

为验证优化效果，使用　　进行文件系统　　性能测试。所有测试均在本地　　虚拟机（　　架构）上运行，单核，　　内存。对比优化前后以及　　基准的性能数据。以下选取部分有代表性的测试项进行对比（单位：　　）：

测试场景	优化前	优化后	
顺序写 （ ）			
顺序读 （ ）			
顺序重写 （ ）			
随机读 （ ）			
随机写 （ ）			
反向读 （ ）			
并发读			
并发写			

表 性能对比（ ，本地文件系统）

测试场景	优化前	优化后	
顺序写 （ ）			
顺序读 （ ）			
顺序重写 （ ）			
随机读 （ ）			
随机写 （ ）			
反向读 （ ）			
并发读			
并发写			

表 性能对比（ ，本地文件系统）

优化前的 在文件系统 吞吐测试中远低于 基准。优化后，顺序写提升约 倍，顺序读提升约 倍，随机读写提升约 倍。优化后的 在部分场景（如顺序重写）已超过 ，在其他场景（如顺序读、随机读）接近 水平。主要优化得益于页缓存容量扩展（从默认配置提升到 ）和统一的 管理策略。

用全局计数和掩码在事件投递入口做早期过滤： 维护全局 计数，计数为零时通知函数立即返回； 维护全局兴趣掩码，掩码对当前事件无命中时跳过后续所有处

理。两处检查都只需一次原子读，无监听时不进入任何路径解析、　　匹配或事件队列操作。一旦用户注册了　　或　　，仍会走完整的匹配和投递流程，功能语义不受影响。

5.9

本章介绍了　　文件系统从底层存储到用户接口的整体设计。挂载子系统支持　　、命名空间隔离与新式挂载　　，　　层通过 `Location` 和 `NodeFlags` 将路径解析与挂载切换解耦，使上层调用方无需感知挂载点跳转。　　子系统在　　基础上引入　　页缓存与元数据缓存，并在写路径、脏页写回、显式预读和　　预热四个维度做了针对性优化，减少小块中的无效读写和元数据开销。文件事件通知提供了　　和　　的完整实现，在无监听者时通过全局计数与兴趣掩码早期过滤，使事件投递逻辑不干扰正常　　路径的性能。

6

信号是　　用于异步事件通知的基本机制。进程之间借助信号传递事件，内核也利用信号将异常、终端中断、子进程状态变化等事件转交给进程处理。

　　基于　　改造而来，信号机制的整体框架沿用了　　已有的设计。在此基础上，　　针对若干语义缺口进行了补充：进程的可暂停状态、信号发送时的权限校验，以及父进程对子进程停止与继续事件的可见性。本章先介绍　　信号机制的整体结构与现存问题，随后逐一说明　　所做的增量。

6.1 StarryOS

6.1.1

　　的信号机制由底层库 starry-signal 提供核心语义，内核负责将其接入任务管理、系统调用以及返回用户态的处理路径。信号管理器按进程和线程两层组织：

进程级：每个进程持有一个进程级信号管理器，保存进程共享的待处理信号队列、信号处理动作表，以及进程内所有线程的引用。当信号发往进程时，会从中挑选一个未屏蔽该信号的线程负责处理。

线程级：每个线程持有一个线程级信号管理器，保存该线程私有的待处理信号、屏蔽集和信号栈。

待处理信号按类型分别存储。标准信号仅保留一份，重复发送会被合并；实时信号按发送顺序排队，确保每次发送都不丢失。

6.1.2

信号检查发生在每次从内核返回用户态之前。内核从待处理队列中取出一个未被屏蔽的信号，根据当前的信号处理动作决定后续行为。可能的动作包括默认终止、默认核心转储、默认忽略、默认停止与继续，以及调用用户注册的信号处理函数。

对于用户处理函数，内核在用户栈上构造一个信号栈帧，其中保存被中断时的寄存器状态、信号信息以及用于恢复执行的上下文，随后将控制流转移到用户处理函数。用户处理函数执行结束后，通过 sigreturn 系统调用回到内核，由内核恢复栈帧并继续原有执行流。

这一发送、入队、检查、递送、恢复的完整链路在　　中已经实现，kill、tkill、tgkill、sigaction、sigprocmask、sigsuspend、sigtimedwait 等系统调用均建立在该链路之上，　　完全沿用。

6.1.3

围绕信号语义的完整性，　　的实现仍存在若干明显缺口：

停止与继续语义未真正实现。　　在收到 SIGSTOP、SIGTSTP 等停止信号时直接令进程退出，对 SIGCONT 则不做任何处理。其结果是进程无法真正暂停，　　的作业控制功能（如 Ctrl-Z、fg、bg）无法工作。

信号发送缺少权限校验。各信号发送接口未对发送者与目标进程之间的身份关系做检查。按　　规范，普通用户只能向同　　的目标发送信号，　　才能向任意进程发送信号，缺少此检查将导致权限隔离失效。

父进程无法感知子进程的状态变化。wait 系列系统调用只能获取子进程的退出状态，对子进程的停止和继续事件无任何通知机制，waitid 的 WSTOPPED、WCONTINUED 语义形同虚设。

6.2

实现停止与继续语义的关键在于：进程需要能够处于 存活但不参与调度 的状态。 中进程仅有运行与退出两种状态，缺少描述 暂停 的中间形态。为此， 引入了一个新的进程等待状态枚举，用以表示进程对外可见的暂停与恢复信息：

```
pub enum ProcessWaitStatus {
    Stopped(Signo),   // 进程被某停止信号挂起，记录触发的信号
    Continued(Signo), // 进程被某继续信号唤醒，记录触发的信号
}
```

代码 进程等待状态

该状态及其配套的事件通道一并挂载到进程的共享数据结构 ProcessData 上：

```
pub struct ProcessData {
    ...
    // 父进程通过 waitid 可见的暂停/继续状态
    pub wait_status: SpinNoIrq<Option<ProcessWaitStatus>>,
    // 被停止线程在此事件上挂起，等待 SIGCONT 或 SIGKILL 唤醒
    pub stop_event: Arc<PollSet>,
}
```

代码 ProcessData 上新增的字段

wait_status 是父进程通过 waitid 系统调用可以读到的状态；stop_event 则承担线程挂起与唤醒的职责，被停止的线程在该事件上阻塞，直到收到 SIGCONT 或 SIGKILL 后被唤醒。

凭借这一对结构， 将信号检查路径中原本空缺的停止与继续两条分支补齐。 check_signals 内的停止与继续动作处理如下：

```
SignalOSAction::Stop => {
    // 写入对外可见的停止状态，并记录触发的信号
    *thr.proc_data.wait_status.lock() =
Some(ProcessWaitStatus::Stopped(signo));
    // 通知父进程的 waitid 调用可以返回
    wake_waiting_parent(&thr.proc_data);
    // 当前线程在 stop_event 上挂起，直至被唤醒
    wait_for_continue(&thr.proc_data);
}
SignalOSAction::Continue => {
    // 状态切回继续，父进程的 waitid 同样需要被通知
    *thr.proc_data.wait_status.lock() =
Some(ProcessWaitStatus::Continued(signo));
    wake_waiting_parent(&thr.proc_data);
}
```

代码 停止与继续动作的处理

进入停止分支后，当前线程在 stop_event 上挂起，直至状态翻转或进程被强制终止。信号发送路径上，识别到 SIGCONT 时将 wait_status 置为 Continued、唤醒 stop_event 上的线程并通知父进程；识别到 SIGKILL 时同样唤醒 stop_event，以确保被停止的进程能够走完退出流程。

该机制使进程首次具备真正意义上的可暂停状态。　　在按下 Ctrl-Z 后能够保留前台作业的完整执行现场，并在 fg 之后从原位置继续执行；　　等调试器所依赖的暂停与恢复语义也得以建立在此机制之上。

6.3

　　对信号发送的权限规定大致为：除　　外，发送者只能向与自己　　或　　匹配的目标发送信号；SIGCONT 例外，同一会话内可以跨　　发送，以支持　　对换过身份的子进程进行作业控制。

　　的 ProcessData 中并未包含任何　　身份相关字段，权限校验缺乏基础。　　为 ProcessData 增补了完整的身份字段，包括真实、有效与保存的　　及　　。在此基础上，在信号发送路径上引入了统一的权限判断逻辑，其规则可概括为：

条件	允许发送
发送者有效　　为　（　）	是
发送者　　或　　与目标　　或　　匹配	是
发送 SIGCONT 且与目标处于同一会话	是
其余情形	否

表　　信号发送的权限规则

　　sys_kill 支持四种目标语义：指定进程、当前进程组、广播以及指定进程组。　　为每种语义都包裹了一层带权限校验的发送过程。对于组发送和广播这类批量操作，校验结果需要进一步区分：只要存在至少一个有权限的目标，操作即视为成功；若找到了目标但全部因权限不足而失败，则返回 EPERM；若完全没有匹配的目标，则返回 ESRCH。

　　身份字段就位后，　　的信号发送行为符合　　权限模型，进程间的隔离得到保障。会话内 SIGCONT 例外规则使　　能够对已通过 setuid 切换身份的子进程发出恢复命令，从而支持完整的作业控制流程。

6.4

　　规定子进程的退出、停止与继续都应通知父进程，以便父进程通过 wait 系列接口获取状态。　　已经具备一个用于子进程退出通知的事件通道（child_exit_event），但仅在子进程退出时被触发，停止与继续两种事件没有相应的唤醒动作，导致 waitid 的 WSTOPPED、WCONTINUED 语义无法生效。

　　没有新建通知通道，而是在停止与继续两条状态变更路径上复用 child_exit_event，使其语义扩展为 子进程任何状态变化均触发 。具体而言，在前述停止与继续的处理过程中，在更新 wait_status 之后统一调用一个辅助函数，由其负责唤醒父进程：

```
fn wake_waiting_parent(proc_data: &ProcessData) {
    // 子进程状态发生变化时，唤醒父进程在 waitid 中的等待
    if let Some(parent) = proc_data.proc.parent()
        && let Ok(parent_data) = get_process_data(parent.pid())
    {
        parent_data.child_exit_event.wake();
    }
}
```

代码　　父进程唤醒

父进程从 `waitid` 中返回时，通过查看 `wait_status` 字段即可区分子进程当前是处于退出、停止还是继续状态。该机制使　　能够在后台作业被停止时立即提示用户，在作业恢复运行时即时更新提示符，避免依赖轮询。前述各项改动也由此真正在用户态可用。

6.5

已经搭建起完整的信号发送、入队、屏蔽与用户处理函数递送链路，　　在此基础上针对若干语义缺口进行了补充：

能力	核心结构	支撑的语义
进程可暂停	`ProcessWaitStatus`、`wait_status`、`stop_event`	作业控制
信号权限	`ProcessData` 上的等身份字段	信号权限模型
父进程可见性	`child_exit_event` 在停止与继续路径上的复用	`waitid` 的 `WSTOPPED`、`WCONTINUED` 语义

表　　对信号机制的补充

网络模块负责组织网络数据的收发与传输。 基于 改造而来，网络栈的整体框架沿用了 已有的设计。 基于 协议栈实现了 的基本功能， 在此基础上针对若干性能瓶颈与兼容性缺口进行了补充。本章先介绍 网络栈的整体结构与现存问题，随后说明 所做的增量。

7.1 StarryOS

7.1.1

已经建立了可工作的网络栈框架。系统基于 协议栈，通过 模块封装 ，向上提供 socket、bind、connect、send、recv 等基础系统调用接口。在设备层， 支持以太网设备和 设备，能够处理 流量。

收发路径的基本框架也已搭建完成。应用程序通过 系统调用发送数据，数据经过协议栈封包后通过设备发送；接收路径通过轮询设备并推进协议栈状态。因此，网络栈面对的不是一片空白，而是主干结构已经成形，但在地址族支持、 配置、 路径优化等方面存在不足的实现。

7.1.2

的网络栈在简单 通信场景下可以工作，但在复杂负载下暴露出若干问题：

地址族支持不完整。 虽然接收 AF_INET 和 AF_INET6 参数，但内部统一按 处理。 地址（如 ::ffff:127.0.0.1）会导致路由查找失败。 等工具在 环境下会混用 、 和 地址，基线无法正确处理这些组合。

Socket 选项语义缺失。缺少 等性能测试工具依赖的关键选项。TCP_INFO 返回固定假值，SO_SNDBUF SO_RCVBUF 不按 语义翻倍，IP_RECVERR、SO_DONTROUTE 等返回 ENOPROTOOPT 导致应用进入异常路径。

Buffer 容量偏小。 和 的发送 接收 均为 ，在高吞吐场景下成为瓶颈。 的 测试仅能达到约 ，远低于预期。

Loopback 路径未优化。本机 127.0.0.1 流量仍走完整的外部网络路径，经过多次拷贝和排队。每次 send() recv() 都无条件推进整个网络栈，累积的轮询开销显著。

7.2

7.2.1 IPv4/IPv6

和 在 中并不是完全独立的两套协议。 允许 接收 连接（通过 地址），也允许通过 IPV6_V6ONLY 选项禁用这一行为。 对地址族的处理过于简化，sys_socket() 虽然接收不同的 domain 参数，但创建的 都按 处理。

问题的关键在于地址格式无法匹配。 测试 ::1（ ）时会创建 并传入 地址，但 内部按 处理，路由查找时发现地址类型不匹配而失败。更复杂的情况是 地址（::ffff:127.0.0.1），它在形式上是 ，但语义上表示 ，需要正确转换才能路由。

为 引入显式的地址族标识。在 TcpSocket 和 UdpSocket 结构中增加 family 字段：

```
pub struct TcpSocket {
    family: IpFamily,  // 标识 IPv4 或 IPv6
```

```
    handle: SocketHandle,
    local_addr: Option<SocketAddr>,
    peer_addr: Option<SocketAddr>,
    ipv6_only: bool,   // 控制 IPv6 socket 是否接受 IPv4-mapped 地址
    // ...
}

pub enum IpFamily {
    Ipv4,
    Ipv6,
}
```

代码 中的地址族标识

创建 时根据 AF_INET 或 AF_INET6 保存对应的 。 和 操作在接收地址参数时，先检查地址类型是否与 匹配。对于 ，若地址是 形式且 ipv6_only 为 ，则提取其中的 部分进行路由；否则按 处理。 在接收连接时也需配合检查。 （::）在 ipv6_only 为 时可以接收 连接（以 形式呈现），但在 ipv6_only 为 时只能接收纯 连接。 在 的匹配逻辑中增加了 和 ipv6_only 的判断，确保连接被正确路由到对应的 。

7.2.2 Socket

和 等性能测试工具在启动时会设置若干 选项以调整行为。 对部分选项的处理不符合 语义，导致工具进入异常路径或直接失败。 针对若干关键选项进行了补齐：

选项	的处理
TCP_INFO	返回包含当前拥塞窗口、 、重传计数等信息的结构体，供 工具诊断连接状态
SO_SNDBUF SO_RCVBUF	按 语义，用户请求的值在内核中翻倍存储；读取时返回 实际分配的值
TCP_QUICKACK	控制 是否立即发送 ，影响延迟敏感场景的响应速度
SO_LINGER	控制 close() 时是否等待未发送数据，以及超时行为
IP_RECVERR	允许应用通过 recvmsg() 获取 错误， 模式依赖 此选项
SO_DONTROUTE	标记数据包不经过路由表，直接发往本地网络

表 补齐的 选项

这些选项的补齐使得 和 在 上的行为与在 上保持一致，不再因缺少选项而回退到低性能路径或报错退出。

7.3 Buffer

原有的 在高吞吐场景下容易成为瓶颈。发送方或接收方频繁因 满而阻塞，限制了吞吐上限。 将 容量提升至适合高吞吐场景的配置：

```
// TCP buffer 从 64KB 提升到 512KB
const DEFAULT_TCP_RX_BUF_LEN: usize = 512 * 1024;
const DEFAULT_TCP_TX_BUF_LEN: usize = 512 * 1024;

// UDP buffer 从 64KB 提升到 1MB
const DEFAULT_UDP_RX_BUF_LEN: usize = 1024 * 1024;
const DEFAULT_UDP_TX_BUF_LEN: usize = 1024 * 1024;

// Router packet buffer 提升到 256 项
const ROUTER_PACKET_BUFFER_LEN: usize = 256;
```

代码 容量配置

扩展后的 使得发送方和接收方不再因容量限制而频繁阻塞，吞吐能力显著提升。

7.4 Loopback

7.4.1

本机流量和外部流量的优化目标完全不同。外部网络需要考虑链路 、拥塞窗口、丢包重传；本机 是内存内投递，不存在物理链路错误，也不应被网络拥塞算法限制。

的 流量仍走完整的网络栈路径： 协议栈封包 协议栈解包，涉及多次拷贝和排队。每次 `send()` `recv()` 都会无条件推进整个网络栈，即使当前 已经有数据可读写，仍然要遍历所有 并轮询底层设备。对于 这种纯内存操作，轮询开销占据了大部分 时间。

7.4.2 Router MTU

在 层面识别 目的地址，对这类包直接投递到 ，跳过设备队列。判断逻辑如下：

```
impl IpAddress {
    pub fn is_loopback(&self) -> bool {
        match self {
            // IPv4: 127.0.0.0/8
            IpAddress::Ipv4(addr) => addr.is_loopback(),
            // IPv6: ::1/128
            IpAddress::Ipv6(addr) => addr.is_loopback(),
        }
    }
}

// Router 发送时判断目的地址
fn route_packet(packet: &Packet) {
    if packet.dst_addr().is_loopback() {
        // 直接投递到 RX buffer, 不经过设备
        self.rx_buffer.push(packet);
    } else {
        // 外部流量走设备队列
        self.device_queue.push(packet);
```

```
    }
}
```

代码 地址判断与直投

同时， 设备使用独立的 配置（ 字节），不受外部网卡 字节限制。大 减少了分片和重组开销，对大数据块传输的性能提升显著。

7.4.3 TCP

协议的拥塞窗口机制用于防止网络过载，但 流量不存在网络拥塞。 的对所有 连接统一应用拥塞控制， 连接的发送速率被人为限制。

在 层面为 添加标记，绕过拥塞窗口检查。发送侧仅受 容量限制，不再受慢启动和拥塞避免算法约束。这一改动使得 吞吐从约 提升至 级别。

7.4.4 UDP

的优化更为激进。由于 无需维护连接状态， 让 完全绕过协议栈封包和路由，直接写入目标 的 ：

```
// UDP socket 发送时判断目的地址
fn send_to(&self, buf: &[u8], addr: SocketAddr) -> Result<usize> {
    if addr.ip().is_loopback() {
        // 直接写入目标 socket 的接收队列
        if let Some(target) = find_local_socket(addr) {
            target.rx_queue.push(buf)?;
            return Ok(buf.len());
        }
    }
    // 外部流量走正常协议栈
    self.inner.send_to(buf, addr)
}
```

代码 直达

这一路径省去了封包、路由查找、解包的全部开销， 性能提升数倍。

7.4.5

在每次 `send()` `recv()` 时都会调用网络栈的 函数，遍历所有 并推进协议栈状态。对于已经有数据的 ，这一轮询是不必要的。

将轮询策略改为按需触发。`send()` `recv()` 先尝试一次真实操作，只有遇到 `WouldBlock`（缓冲区空或满）时才推进网络栈并进入等待。批量操作场景下，多次连续的 `send()` 只在最后一次遇到阻塞时才触发一次 ，而不是每次都 。

`poll()` `select()` `epoll_wait()` 等 多路复用接口在检查 前统一推进一次网络栈，避免每个 重复推进。这一改动在 场景下减少了约 的 调用次数。

7.5

为验证优化效果，使用 和 在 接口上进行性能测试。所有测试均在本地 虚拟机（ 架构）上运行，单核， 内存。对比优化前后以及 基准的性能数据。

7.5.1 iperf3

测试场景	优化前	优化后	
基本测试（ ）			
并发测试（ ）			
反向测试（ ）			
基本测试（ ）			
并发测试（ ）			
反向测试（ ）			

表 性能对比（ ，本地 ）

测试场景	优化前	优化后	
基本测试（ ）			
并发测试（ ）			
反向测试（ ）			
基本测试（ ）			
并发测试（ ）			
反向测试（ ）			

表 性能对比（ ，本地 ）

7.5.2 netperf

测试场景	优化前	优化后	
流式传输			
流式传输 传输率 秒			
请求响应 传输率 秒			
请求响应 传输率 秒			
连接请求响应 传输率 秒			

表 性能对比（ ，本地 ）

优化前的 在 吞吐测试中远低于 基准，主要原因是 流量仍走完整网络栈路径且受拥塞窗口限制。优化后， 吞吐提升约 倍， 吞吐提升约 倍，已接近或部分超过 在同等测试环境下的表现。

从约 提升至 ，与 的 基本持平。

7.6

已经搭建起基于 的网络栈框架，提供了 的基本功能。在此基础上针对若干性能瓶颈与兼容性缺口进行了补充：

方面		增量
地址族	仅	区分 ，支持 和 IPV6_V6ONLY
选项	缺少关键选项	补 齐 TCP_INFO、 SO_*BUF、 TCP_QUICKACK 等
容量	均	， ， 项
优化	走完整网络栈	直投、大 、绕过拥塞窗口、 直达、按需轮询

表 对网络模块的补充

这些增量使得 能够正确运行 、 等网络性能测试工具。 吞吐从约 提升至 级别，外部网络性能保持不变。

8 QEMU

初赛阶段在两种 虚拟机上运行： （qemu-system-riscv64 -machine virt）和 （qemu-system-loongarch64）。两种架构的页表格式、异常编码、中断控制器和设备总线完全不同。把这些差异直接暴露给上层，调度器、内存管理器和系统调用处理代码就会散布架构条件分支。本章说明 如何通过三层平台抽象将差异封死在平台实现层， 则在两种架构上提供行为一致的设备接口。

8.1

8.1.1

将硬件差异分三层处理，每层只吸收它自己那一层的差异。

第一层是 axcpu，处理 本身的寄存器和异常机制。axcpu/src/riscv/ 实现 的上下文保存与恢复、用户态进入与返回；axcpu/src/loongarch64/ 做同样的事，用的 era badv 等 。两边向上提供相同的 TrapFrame、context_switch、uspace 接口。

第二层是 axplat，用 定义平台接口：IrqIf（中断注册与分发）、TimeIf（时钟读取与单次定时器）、MemIf（物理内存区域描述）、ConsoleIf（串口输出）。axplat-riscv64-qemu-virt 和 axplat-loongarch64-qemu-virt 各自实现这些 ，通过 #[impl_plat_interface] 宏完成编译期静态绑定。

第三层是 axhal，把 axplat 调用包装成普通函数，供内核其余部分使用。调度器、内存管理器和驱动框架调用的是 axhal::time::wall_time()、axhal::irq::register() 这类接口，不直接触碰任何 特有寄存器。

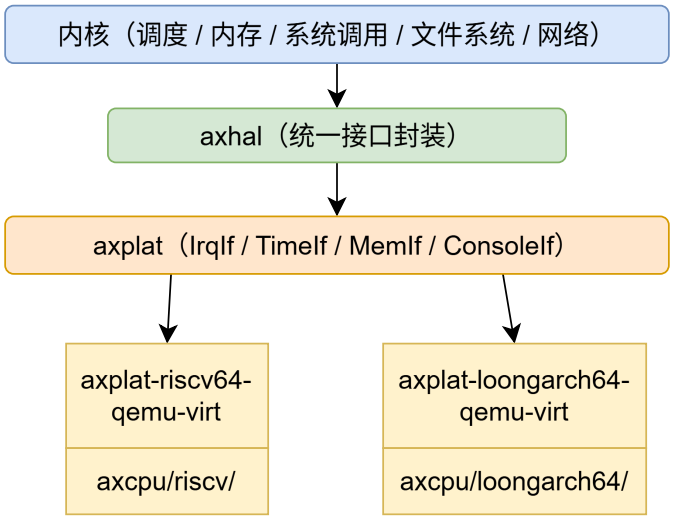


图 平台抽象三层结构

8.1.2

的外部设备中断经 路由。scause 区分定时器（S_TIMER）、核间中断（S_SOFT）、外部中断（S_EXT）三类。处理外部中断时需先向 获得设备 号，处理完后再确认。

的路由链不同：设备中断由 收集后送入 ，再由 向处理器核心报告；定时器中断通过 ecfg 寄存器的 LineBasedInterrupt::TIMER 位独立控制。

两套机制都实现了同一个 IrqIf ：

方法	实现	实现
set_enable	操作 sie 位或 寄存器	操作 ecfg.lie 位或 使能寄存器
register	按 scause 值分别注册	按 TIMER_IRQ EIOINTC_IRQ 分类注册
handle	外部中断时 得到真实 号，再调度	读 状态位后调度
send_ipi	通过 send_ipi() 指定 发送	通过 功能发送

表 在两种架构下的实现：接口相同，硬件路径不同

驱动框架调用 axplat::irq::register(irq, handler) 时，对 还是 一无所知。

8.1.3

通过 （sbi_time::set_timer）设置定时器，time::read64() 读取当前计数；使用 内置的稳频计数器，tcfg tval 寄存器控制定时器触发。两者同样通过 TimeIf 统一：current_ticks() 读各自的硬件计数器，ticks_to_nanos() 完成频率转换，set_oneshot_timer() 设置下一次中断。调度器的时间片计算和 clock_gettime 系统调用只操作 axhal::time::monotonic_time()，不感知底层计数器差异。

8.2 QEMU

资源		
启动方式	， 内核直接加载	， 内核直接加载
页表格式	三级页表，satp	四级页表，PGDL PGDH （见第 章）
中断控制器	， 0x0c00_0000	
定时器	，mtime mtimecmp	稳频计数器，tcfg tval
设备总线	（固定地址范围枚举）	（ 配置）
串口	，	，

表 两种 平台的关键硬件资源对比

页表格式的差异已在第 章展开，本章不重复。

8.3 VirtIO

8.3.1

赛方两种平台均提供两块 磁盘（x0 承载文件系统，x1 为额外测试磁盘）和一张网卡，区别仅在于设备挂载的总线不同。

上， 机型将 设备寄存器映射到固定 地址范围。axdriver 初始化时遍历该范围，将物理地址转换为内核虚拟地址后调用 probe_mmio_device() 探测设备类型。

上， 设备挂在 总线上。axdriver 通过 枚举 ，匹配厂商 0x1af4，按设备 区分块设备（0x1001 0x1042）和网卡（0x1000 0x1041），配置 后调用 probe_pci_device() 初始化传输层。

两条路径都以 VirtIoTransport 为参数进入 D::try_new(transport, irq?)，其中 D 是实现了 VirtIoDevMeta 的具体类型（VirtIoBlk 或 VirtIoNet）。成功初始化的设备通过 AxDeviceEnum 注册到全局设备表，文件系统挂载和网络协议栈直接取用，不感知底层总线差异。

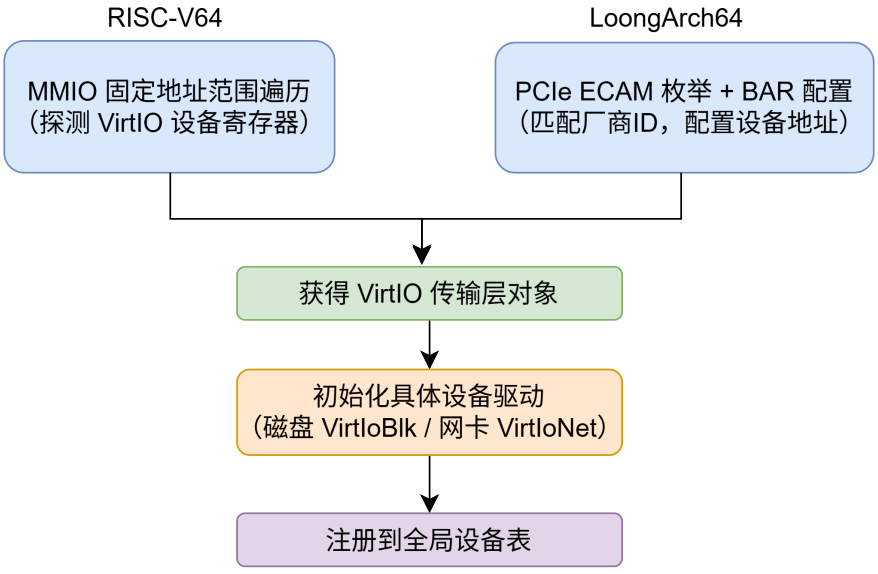


图 设备发现路径

8.3.2 VirtIO DMA

协议以 为数据通道：驱动将控制信息和数据缓冲区组织为描述符链，写入可用队列（ ）并通知 ； 完成后将结果写入已用队列（ ），驱动据此回收描述符。块设备用它提交扇区读写，网卡用它收发报文，两者共用同一套队列管理模型。

描述符中的缓冲区地址必须是物理地址。VirtIoHalImpl 实现 VirtIoHal ，负责 页面的分配与地址转换：

```
// axdriver/src/virtio.rs
unsafe impl VirtIoHal for VirtIoHalImpl {
    fn dma_alloc(pages: usize, _dir: BufferDirection) -> (PhysAddr,
NonNull<u8>) {
        let Ok(vaddr) = global_allocator().alloc_pages(pages, 0x1000,
UsageKind::Dma)
        else { return (0, NonNull::dangling()) };
        let paddr = virt_to_phys(vaddr.into());
        (paddr.as_usize(), NonNull::new(vaddr as _).unwrap())
    }
    unsafe fn dma_dealloc(_paddr: PhysAddr, vaddr: NonNull<u8>, pages: usize) -
> i32 {
        global_allocator().dealloc_pages(vaddr.as_ptr() as usize, pages,
UsageKind::Dma);
        0
    }
    unsafe fn mmio_phys_to_virt(paddr: PhysAddr, _size: usize) -> NonNull<u8> {
```

```
        NonNull::new(phys_to_virt(paddr.into()).as_mut_ptr()).unwrap()
    }
}
```

代码 `VirtIoHalImpl` 的 分配与地址转换，两种架构共用同一份实现 `dma_alloc` 从全局分配器申请物理连续内存并将物理地址填入描述符；请求结束后 `dma_dealloc` 释放这些页面，避免设备访问已回收的缓冲区。 寄存器通过 `mmio_phys_to_virt` 映射后访问。这套接口在 和 上完全相同。

8.4

是 差异的终止线。内核核心模块只调 `axhal` 的接口，不直接操作 或 ，不读取 `mtime` 或 稳频计数器，也不关心 设备在 还是 总线上。平台实现层把硬件差异翻译成统一接口，差异到此为止，不再向上传递。

赛方为两种架构提供相同的设备拓扑，内核在两种平台上经历相同的初始化序列和 路径。用户态进程看到的文件系统语义、网络行为和时钟语义不因底层 而不同。